RW BLE Android SDK User Manual

1. Introduction

This document mainly explains the functional interfaces and usage scenarios provided in the SDK.

This document is only applicable to RW's Bluetooth devices.

1.1 Applicable platforms and languages

- Android 7.0 and above (API 24 and above), language Kotlin.

1.2 Related terms

- App: This article refers to applications running on mobile phones or tablets;
- Device: This article refers to wearable hardware devices: such as watches, rings, etc.;

1.3 Precautions

1. It is best to use this SDK in conjunction with the sample project `RW_SDK_DEMO`; for reference, just focus on the two pages of `NewMainActivity` and `ScanActivity`.
2. Most of the command operations of `DHBleSdk` are implemented by subscribing to the corresponding callback `DHBleSdk.subscribeData()`; when no longer in use, please unsubscribe `DHBleSdk.dispose()`;

2. Quick Start

Step 1: Get the latest version of Android Studio.

To use the RW BLE SDK for Android in your development project, you need to install Android Studio.

Step 2: Add the SDK and its dependencies.

Choose exactly one of the two RW SDK AAR variants. Choose the nordic variant only if you need Nordic OTA support. Otherwise, choose basic:

Variant	Scenario	Additional dependency
basic	No Nordic OTA	None
nordic	Requires Nordic OTA updates	mcumgr-ble:3.3.1

Copy the selected RW SDK AAR into `app/libs/`. Add only the matching dependency block below to `dependencies { ... }` in `app/build.gradle` (Groovy). Replace the example filenames with the names of your downloaded files:

basic:

```
implementation files('libs/blesdk-rwfit-release_v2_261008.aar')
```

nordic:

```
implementation files('libs/blesdk-rwfit-nordic-release_v2_261008.aar')
implementation 'no.nordicsemi.android:mcumgr-ble:3.3.1'
```

> **Note**
>
> Build requirements: basic requires `compileSdk` 34 or higher; nordic requires `compileSdk` 37 or higher. A verified reference configuration for nordic is AGP 9.1.1 / Gradle 9.3.1 / Kotlin 2.4.20 / compileSdk 37. The minimum supported Android version remains Android 7.0 (API 24).

Step 3: You need to enable Bluetooth on your phone and grant Bluetooth and location permissions.

```
    private fun getBluePermissions(): List<String> {
        val permissions = mutableListOf<String>()
        permissions += Permission.BLUETOOTH_SCAN
        permissions += Permission.BLUETOOTH_ADVERTISE
        permissions += Permission.BLUETOOTH_CONNECT
        permissions += Permission.ACCESS_COARSE_LOCATION
        permissions += Permission.ACCESS_FINE_LOCATION
        return permissions
    }
```

Step 4: Initialize the SDK

```
DHBleSdk.initSDK(this)
```

> **Caution**
>
> The SDK outputs and saves Bluetooth logs by default. Log files are stored in the app's internal `files/logger/devices/` directory. Call `DHBleSdk.setLogEnable(false)` to disable both log output and local file writing.

SDK obfuscation

The release AAR already obfuscates the SDK's internal implementation and includes consumer rules that preserve its public API. When the AAR is integrated normally through Gradle, these rules are merged automatically into the application's R8/ProGuard configuration, so no additional rules are normally required.

If the AAR is repackaged in a way that discards its consumer rules, add the following fallback rules to the application's ProGuard configuration:

```
-keepattributes Signature,*Annotation*,InnerClasses,EnclosingMethod

-keep class com.example.blesdk.DHBleSdk { *; }

-keep class com.example.blesdk.ble.** { *; }
-keep class com.example.blesdk.rwbaselibrary.HandlerCallback { *; }

-keep class com.example.blesdk.bean.** { *; }
-keep class com.example.blesdk.callback.** { *; }

-keep class com.example.blesdk.blering.BaseDataCallback { *; }
-keep class com.example.blesdk.blering.DeviceDataCallback { *; }
-keep class com.example.blesdk.blering.BaseAnswerCallback { *; }
```

```
-keep class com.example.blesdk.blering.RingConnectBleCallback { *; }
-keep class com.example.blesdk.blering.RingBleError { *; }
-keep class com.example.blesdk.blering.OnDevicePushListener { *; }
-keep class com.example.blesdk.blering.PushData { *; }
-keep class com.example.blesdk.blering.DevicePushType { *; }

-keep class com.example.blesdk.utils.** { *; }
```

3. API Reference

3.1 Device search and connection, binding and reconnection.

3.1.1 Search for Bluetooth devices

> Interface description: To search for Bluetooth devices, you need to initialize `ScanBleService` first and implement the `ScanDeviceCallback` callback interface.

```
// 1. Initialize and register the callback.
ScanBleService.getService().initBle(this)
ScanBleService.getService().registerScanBleCallback(this)

// 2. Start searching
ScanBleService.getService().startScan(true,null)

//3. The ScanDeviceCallback interface will provide callbacks for the discovered Bluetooth devices.
public interface ScanDeviceCallback {
  public void onScanDevice(BleDevice device);
  public void onScanFinish();
  public void onError(int errorCode,Exception e);
}

//4. Unregister callback
ScanBleService.getService().unRegisterScanBleCallback()
```

Main properties of the `BleDevice` scan result:

BleDevice property	Type	Description
bleName	String	Device Bluetooth name
bleMac	String	Device Bluetooth MAC address
bleRssi	int	Signal strength at scan time, in dBm
bleDeviceId	String	Device model identifier, not a unique device serial number; empty if unavailable
batteryStatus	int	Charging status: `0` not charging, `1` charging, `2` full; `-1` unavailable or unsupported. Requires device firmware support
deviceForm	int	Product form: `0` ring without a screen, `1` ring with a screen, `2` bracelet without a screen, `3` bracelet with a screen, `4` wristband without a screen, `5` wristband with a screen, `6` watch; `-1` unavailable or unsupported. Requires device firmware support

> `batteryStatus` is not a battery percentage. Read the battery level after connecting; see 3.2.1.4 Get Battery Level.

3.1.2 Stop searching

> Interface description: Stop searching for Bluetooth devices.

```
ScanBleService.getService().stopScan()
```

3.1.3 Device connection and status monitoring

> Interface description: Connects to the specified device and monitors the device's connection status.

```
// 1. Initialize and register the callback.
DHBleSdk.setConnectBleCallback(this)

// 2. Connect devices
DHBleSdk.connectDeviceWithModel(bleDevice)

// 3. Implement and receive callbacks for Bluetooth connection status.
interface RingConnectBleCallback {
  fun onRingConnecting(device: BleDevice?)
  fun onRingConnected(device: BleDevice?)
  fun onRingConnectFailed(device: BleDevice?, reason: RingBleError = RingBleError.UNKNOWN)

  fun onRingDidFunctionMenu(device: BleDevice?, supportMenuBean: SupportMenuBean)
}
```

`RingConnectBleCallback` Interface Description:

Method	Description
onRingConnecting	Connecting
onRingConnected	After calling `connectDeviceWithModel`, the function will return upon successful connection.
onRingConnectFailed	Called when connection fails or disconnects. For password authentication failure, `reason` is `PASSWORD_AUTH_FAILED`.
onRingDidFunctionMenu	The function will return after successfully obtaining the device configuration table; business operations should be performed after this point.

> **Tip**
>
> After connecting, business operations should only be performed after the `onRingDidFunctionMenu` event.

3.1.4 Disconnect the device.

> Interface description: Disconnect the currently connected device and monitor the device connection status.

```
DHBleSdk.disconnect()
```

> After calling this method, wait for `onRingConnectFailed` before reconnecting or binding another device, so the current connection and related resources can be fully released.

3.1.5 Local binding and automatic reconnection, unbinding

> **Important**
>
> Android local binding requires manual implementation of automatic reconnection; this SDK does not provide this functionality. After a successful connection, you can save the MAC address and necessary configuration information for convenient display and reconnection on the user interface.

3.1.6 Equipment Function Configuration Table

> **Important**
>
> Due to the variety of device models and their differing supported features, a feature list has been introduced to allow users to check the supported features of each device. Please refer to the `SupportMenuBean` class for details. You can save the feature list content according to your specific business requirements.

`fun onRingDidFunctionMenu(supportMenuBean:SupportMenuBean)`

DeviceFuncV2Model class attribute definitions:

SupportMenuBean attribute	Description
isPushMsgEnableSwitch	Enable or disable the message control switch?
pushMsgSwitchValue	Supported message types, low 32 bits (bit0-bit31)
pushMsgSwitchValue2	Supported message types, high 32 bits (bit32-bit63); defaults to 0 on old devices

activityDataInterval	Today's step-detail interval in minutes; an unconfigured value is normalized to 60
isAlarm	Does it support an alarm clock?
isBrightScreenSleepTime	Does it support screen sleep time settings?
isBrightScreenTime	Does it support screen-on time adjustment?
isNewSport	Does it support multiple sports modes?
isRememberSwitch	Does it support enabling/disabling Muslim prayers?
isSupportHrReminder	Does it support HR (heart rate) alarm notification function?
isSupportBoReminder	Does it support SpO2 alarm notification functionality?
isSupportMotoVibrationLevel	Does it support motor vibration alerts?
isSupportAlarmVibrationDuration	Does it support alarm vibration duration setting?
isSupportVibrationInterval	Does it support vibration interval setting?
isStep	Does it support step counting?
isHr	Does it support heart rate monitoring?
isBloodPress	Does it support blood pressure measurement?
isSleep	Does it support sleep mode?
isBloodOxy	Does it support blood oxygen monitoring?
isHrv	Does it support heart rate variability?
isPressure	Does it support pressure?
isBloodSugar	Does it support blood glucose monitoring?
isMuslimCountData	Do you support praise?
isDataTypeTemperature	Does it support temperature monitoring?
isSupportMuslimTimeDisplayMode	Does it support Muslim time display mode?
isSupportSensorRawPPG	Does it support PPG raw data?
isSupportPPGMonitoring	Does it support PPG timed monitoring?
isSupportTemperatureMonitoring	Does it support temperature timed monitoring?
isSupportCountReminder	Does it support count reminder interval setting?
isSupportSensorRawACC	Does it support ACC raw data?
isSupportSensorRawPPGRed	Does it support PPG Red raw data?
isSupportRecording	Does it support AI recording?
isSupportDevicePasswordAuth	Does it support device password authentication?

isSupportDeviceChallenge	Does it support device identity authentication (HMAC-SHA256 challenge-response)?
isSupportSedentary	Does it support sedentary reminder setting?
isDrink	Does it support drinking reminder setting?
isSupportSensorRawIR	Does it support IR (infrared) raw data?
isSupportSensorRawSleep	Does it support sleep real-time data?
isSupportFallDetect	Does it support fall detection alert?
isSupportRecording	Does it support AI recording?
isSupportDevicePasswordAuth	Does it support device password authentication?
isSupportScreenControl	Does it support instant screen on/off control?
isSupportUnitSetting	Does it support metric/imperial unit settings?
isSupportVibrationControl	Supports custom vibration control
isSupportCustomNumberDisplay	Supports custom number display
isSupportCountControl	Supports counting recognition control and state queries
isSupportPpgSampleRate	Supports setting and reading the PPG sample rate
isSupportCustomerModel	Supports setting and reading the customer model
isSupportCompositeMeasurement	Supports composite health measurement (3.2.2.1)
isSupportCustomTextDisplay	Supports custom text display and duration settings (3.2.1.35)

3.1.7 Read connected RSSI

`DHBleSdk.readRssi(callback: RssiCallback)` returns signal strength in dBm through `onRssiRead(rssi: Int)`, or an error through `onRssiFailed(errorCode: Int)`. One-shot callbacks run on the main thread; no subscription or `dispose()` is required.

Error constant in RssiCallback	**Meaning**
`ERROR_NOT_CONNECTED` (-1)	Not connected or disconnected
`ERROR_BUSY` (-2)	RSSI request pending or OTA active
`ERROR_START_FAILED` (-3)	System refused to start reading
`ERROR_TIMEOUT` (-4)	No result within 5 seconds of starting
Other nonzero values	Android GATT failure status

```
DHBleSdk.readRssi(object : RssiCallback {
    override fun onRssiRead(rssi: Int) {
        // Update signal strength in dBm
    }

    override fun onRssiFailed(errorCode: Int) {
        // Handle failure; skip this reading on ERROR_BUSY
    }
})
```

3.2 Device function operation

3.2.1 Basic function command interface

3.2.1.1 Get SDK Version

> Get the SDK version number.

Method Description:

`DHBleSdk.getSDKVersion()`

Example of usage:

```
Log.e("RWSDK", DHBleSdk.getSDKVersion())
```

3.2.1.2 Set user information

> User information settings are related to step count, calories burned, and distance. During device initialization, the default settings are: gender 1, age 18, height 170cm, and weight 65 kg.
>
> Subscribe to `CommonStatusCallback` to receive the results.

Method Description:

`fun setUserInfo(personBean: PersonBean)`

Parameter Description:

parameter	type	Description	
personBean	PersonBean	class	gender: Gender (0. Female, 1. Male) height: Height in cm, floating-point number weight: Weight in kg, floating-point number age: Age

Example of usage:

```
DHBleSdk.subscribeStatus(object : CommonStatusCallback{
  override fun onSuccess(id: Int) {
    Log.e("RWSDK", "user info set ok")
  }

  override fun onFail(id: Int, errorCode: Int) {
    Log.e("RWSDK", "user info set failed")
  }
```

```
})


val persionBean = PersonBean()
persionBean.measureUnit = 0
persionBean.gender = 1
persionBean.height = 170.5f
persionBean.weight = 80f
persionBean.age = 20
DHBleSdk.setUserInfo(persionBean)
```

3.2.1.3 Get Device Information

> Get the device model, firmware version, screen information, UI version, and product form.
>
> Subscribe to `FirmwareCallback` to receive the result.

Method Description:

`fun getFirmwareVersionJL()`

Return Value:

FirmVersionBean property	Type	Description
deviceClazz	String	Device model, the unique identifier for each product model
deviceNo	String	Firmware version
screenType	int	Screen type: 0. square, 1. round
screenWidth	int	Screen width
screenHeight	int	Screen height
uiVersion	String	UI version
deviceForm	int	Product form: 0 ring without a screen, 1 ring with a screen, 2 bracelet without a screen, 3 bracelet with a screen, 4 wristband without a screen, 5 wristband with a screen, 6 watch; -1 unavailable or unsupported
customerModel	String	Customer model; defaults to the device model and returns the device model when unavailable. See 3.2.1.34 for setting and reading this value

> **Important:** Before a firmware upgrade, verify that the device's `deviceClazz` matches the device model supported by the firmware package. Proceed with the upgrade only when they match. Do not upgrade if they do not match.

Example of usage:

```
DHBleSdk.subscribeData(object : FirmwareCallback {
  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "firmware info get failed, errorCode=$errorCode")
  }
```

```
  override fun onResult(data: FirmVersionBean?) {
    data?.let {
      Log.e("RWSDK", "firmware info --> $it")
    }
  }
})

DHBleSdk.getFirmwareVersionJL()
```

3.2.1.4 Get Battery Level

> Get the device's battery level.
>
> Subscribe to `PowerCallback` to receive the result.

Method Description:

`fun getPowerJL()`

Return Value:

PowerBean property	Type	Description
power	int	Remaining battery level, 0-100
powerStatus	int	Charging status: `0` not charging, `1` charging; requires device firmware support. Defaults to `0` when not returned by older firmware.

Example of usage:

```
DHBleSdk.subscribeData(object : PowerCallback {
  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "power get failed, errorCode=$errorCode")
  }

  override fun onResult(data: PowerBean?) {
    data?.let {
      Log.e("RWSDK", "power --> $it")
    }
  }
})

DHBleSdk.getPowerJL()
```

3.2.1.4.1 Real-time Battery Monitoring

Supported devices actively push the current battery level and charging status when charging starts or stops. This capability requires device firmware support and is not a periodic update. Call `getPowerJL()` when the app needs to actively query the current battery level.

The push type is `DevicePushType.POWER` and `value` is a `PowerBean` (same entity as the `getPowerJL()` query result). See 3.2.1.21 Device Push Notification for how to subscribe.

3.2.1.5 Getting and Setting Video Control Switch

> Sets whether to enable ring gesture control for watching videos; This function requires Bluetooth HID pairing. HID pairing can be done using `BlueToothUtils createOrRemoveBond` (type 1 for pairing, 2 for unpairing), or you can implement it yourself.
>
> Subscribe to `videoHidCallback` to get the result.

Method Description:

`fun setVideoHidJL(videoHidBean: VideoHidBean)`

Parameter Description:

Parameter	Type	Description	Value
videoHidBean	VideoHidBean	Class	hidOpen: 0. Off 1. Video On 2. Book 3. Music

Example of usage:

```
// Set video control switch
DHBleSdk.subscribeData(videoHidCallback)
val videoHidBean = VideoHidBean()
videoHidBean.hidOpen = 1  //Whether to open short video control
DHBleSdk.setVideoHidJL(videoHidBean)

// Get video control switch
DHBleSdk.subscribeData(videoHidCallback)
DHBleSdk.getVideoHidJL()
```

> `VideoHidCallback` only returns the result of setting or reading the control mode. When music mode is enabled with `hidOpen = 3`, also subscribe to `MusicPushSettingCallback` to receive music control commands actively reported by the device.

```
val musicPushCallback = object : MusicPushSettingCallback {
    override fun onResult(data: Int?) {
        when (data) {
            3 -> { /* Previous */ }
            4 -> { /* Next */ }
        }
    }

    override fun onFail(errorCode: Int) {}
    override fun onSuccess() {}
}

DHBleSdk.subscribeData(musicPushCallback)
val musicMode = VideoHidBean().apply { hidOpen = 3 }
DHBleSdk.setVideoHidJL(musicMode)

// Remove the listener when music control events are no longer needed.
DHBleSdk.dispose(musicPushCallback)
```

3.2.1.6 Getting and Setting LED Screen Brightness

> Configuration table attribute: `isLEDLight`;

> Subscribe to `BrightLedLevelCallback` to get the result.

Method Description:

`fun setRingLedLevel(brightScreenBean: BrightScreenLedBean)`

Parameter Description:

Parameter	Type	Description	Value
brightScreenBean	BrightScreenLedBean	Class	isOpen: false for off, true for (1-3 Level) lcdLevel: 1 Dim light, 2 Soft light, 3 Strong light

Example of usage:

```
// Get LED screen brightness
DHBleSdk.subscribeData(brightLedLevelCallback)
DHBleSdk.getRingLedLevel()

// Set LED screen brightness
DHBleSdk.subscribeData(brightLedLevelCallback)
val tBrightScreenLedBean = BrightScreenLedBean()
tBrightScreenLedBean.isOpen = true //false is off, true is (1-3Level)
tBrightScreenLedBean.lcdLevel = 3 //1-3Level: 1 low 2 mid 3 high
DHBleSdk.setRingLedLevel(tBrightScreenLedBean)
```

3.2.1.7 Get and Set Wearing Position

> Get or set the hand on which the ring is worn.
>
> Function table property: `isWearDir`.
>
> Subscribe to `WearHandCallback` to receive the result.

Method Description:

`fun getRingWearDir()`

`fun setRingWearHand(isOpen: Boolean)`

Parameter Description:

Parameter	Type	Description	Value
isOpen	Boolean	Wearing position	false. left hand, true. right hand

Return Value:

FactoryInBean property	Type	Description
isOpen	int	Wearing position: 0. left, 1. right

Example of usage:

```
val wearHandCallback = object : WearHandCallback {
  override fun onSuccess() {
    Log.e("RWSDK", "wear hand operation success")
```

```
    }

    override fun onFail(errorCode: Int) {
      Log.e("RWSDK", "wear hand operation failed, errorCode=$errorCode")
    }

    override fun onResult(data: FactoryInBean?) {
      data?.let {
        Log.e("RWSDK", "wear hand=${it.isOpen()}")
      }
    }
}

DHBleSdk.subscribeData(wearHandCallback)

// Get wearing position
DHBleSdk.getRingWearDir()

// Set wearing position to the left hand
DHBleSdk.setRingWearHand(false)

// Set wearing position to the right hand
DHBleSdk.setRingWearHand(true)
```

3.2.1.8 Start and Stop Photo Taking

> The APP must implement its own camera page, for example with a third-party camera library. Launching the phone's system camera does not provide ring-controlled capture. The SDK controls the camera mode and delivers capture events; the APP takes photos and saves them to the phone's gallery.
>
> Function table property: `isTakePhoto`.
>
> Subscribe to `TakePhotoCallback` to receive photo notifications from the device.

Usage flow:

1. When entering the APP's camera page, subscribe to `TakePhotoCallback` and call `DHBleSdk.controlTakePhotoJL(1)` to put the ring into camera control mode.
2. When the user triggers a capture gesture on the ring, `onResult()` receives `data=2`. Use the camera library to take a photo and save it to the phone's gallery.
3. When leaving the camera page, call `DHBleSdk.controlTakePhotoJL(0)` to exit camera control mode and unsubscribe.

Method Description:

`fun controlTakePhotoJL(controlType: Int)`

Parameter Description:

Parameter	Type	Description	Value
controlType	Int	Photo control	0. disable photo, 1. enable photo

Return Value:

Callback data	Type	Description
data	Int	2. device notifies the APP to take a photo

Example of usage:

```
private val takePhotoCallback = object : TakePhotoCallback {
  override fun onSuccess() {
    Log.e("RWSDK", "take photo control success")
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "take photo control failed, errorCode=$errorCode")
  }

  override fun onResult(data: Int?) {
    if (data == 2) {
      // Use the APP's camera library to take a photo and save it to the phone's gallery.
    }
  }
}

// Call when the APP enters the camera page.
fun onCameraPageOpened() {
  DHBleSdk.subscribeData(takePhotoCallback)
  DHBleSdk.controlTakePhotoJL(1)
}

// Call when the APP exits the camera page.
fun onCameraPageClosed() {
  DHBleSdk.controlTakePhotoJL(0)
  DHBleSdk.dispose(takePhotoCallback)
}
```

3.2.1.9 Finding the Device

> After calling the find function, the device's light or screen will light up.

Method Description:

`fun controlFindDeviceJL()`

Example of usage:

```
DHBleSdk.controlFindDeviceJL()
```

3.2.1.10 Shutdown, Factory Reset

> Subscribe to `DeviceControlCallback` to get the result.

Method Description:

`fun setPowerOffJL(type: Int)`

Parameter Description:

Parameter	Type	Description	Value
type	Int	Integer	Shutdown: Constants.CONTROL_DEVICE_POWER_OFF Factory Reset: Constants.CONTROL_DEVICE_RECOVERY

Example of usage:

```
// Shutdown
DHBleSdk.setPowerOffJL(Constants.CONTROL_DEVICE_POWER_OFF) //Shutdown

// Factory Reset
DHBleSdk.setPowerOffJL(Constants.CONTROL_DEVICE_RECOVERY) //Factory Reset

// You can subscribe to the DeviceControlCallback callback
```

3.2.1.11 Alarm Clock

3.2.1.11.1 Getting Set Alarms

> Configuration table attribute: `isAlarm`
>
> Subscribe to the `AlarmCallback` callback;

Method Description:

`DHBleSdk.getAlarmRemindJL();`

Example of usage:

```
// Get the alarms saved on the device
DHBleSdk.subscribeData(alarmCallback)
DHBleSdk.getAlarmRemindJL()
```

3.2.1.11.2 Setting Alarms

> **The current protocol does not support modifying alarms individually. Any operation to switch on/off or delete a single alarm requires resending all alarm configurations.** **
>
> Subscribe to the `AlarmCallback` callback;

Method Description:

`fun setAlarmRemindJL(reminderBeans: List<AlarmRemainderBean>)`

Parameter Description:

Parameter	Type	Description	Value
reminderBeans	List	Alarm array	isOpen: true/false repeatModel: IntArray(7) Sunday to Saturday, set to 1 for days to repeat startHour: Alarm start hour startMin: Alarm start minute alarmTag: Set to an empty string;

Example of usage:

```
// Set alarm
```

```
DHBleSdk.subscribeData(alarmCallback)

val params = mutableListOf<AlarmRemainderBean>()
val alarmRemainderBean = AlarmRemainderBean()
alarmRemainderBean.alarmTag = ""
alarmRemainderBean.repeatModel = IntArray(7) // Single time; Sunday to Saturday, set to 1 for days to
repeat
alarmRemainderBean.startHour = 7
alarmRemainderBean.startMin = 0
alarmRemainderBean.isOpen = true
alarmRemainderBean.alarmId = 0
params += alarmRemainderBean

val alarmRemainderBean2 = AlarmRemainderBean()
alarmRemainderBean2.alarmTag = ""
alarmRemainderBean2.repeatModel = IntArray(7) // Single time; Sunday to Saturday, set to 1 for days to
repeat
alarmRemainderBean2.startHour = 8
alarmRemainderBean2.startMin = 0
alarmRemainderBean2.isOpen = false
alarmRemainderBean2.alarmId = 0
params += alarmRemainderBean2

DHBleSdk.setAlarmRemindJL(params)
```

3.2.1.11.3 Delete All Alarms

> `DHBleSdk.deleteAllAlarmRemindJL`;
>
> Subscribe to the `AlarmCallback` callback;

Method Description:

`DHBleSdk.deleteAllAlarmRemindJL`

```
//Subscribe to callback
DHBleSdk.subscribeData(alarmCallback)
//Delete all alarms
DHBleSdk.deleteAllAlarmRemindJL()
```

3.2.1.12 Vibration Count Setting and Getting

> Configuration table attribute: `isSupportMotoVibrationLevel`
>
> Device vibration count;
>
> Subscribe to the `VibrationCountCallback` callback.

Method Description:

`fun setVibrationCount(level:Int, count: Int)`

`fun getVibrationCount()`

Parameter Description:

Parameter	Type	Description	Value
level	Int	Integer	Vibration intensity, 0: Off 1: Low 2: Medium 3: High; *Can be ignored if this function is not defined*
count	Int	Integer	The number of vibrations can be set (0-6 times), initial default is 2 times. Setting to 0 means no vibration

Example of usage:

```
//Setting
DHBleSdk.subscribeData(vibrationCountCallback)
DHBleSdk.setVibrationCount(1, 2) //Vibration count 2 times

//Getting
DHBleSdk.subscribeData(vibrationCountCallback)
DHBleSdk.getVibrationCount()
```

3.2.1.13 Screen Sleep Mode Setting and Getting

> Sets screen sleep on/off and time;
>
> Configuration table attribute: `isBrightScreenSleepTime`
>
> Subscribe to `BrightTimeCallback` to get the result.

Method Description:

`fun setRingBrightScreenSleepTime(briScreenTime: BrightScreenTimeBean)`

`fun getRingBrightScreenSleepTime()`

Parameter Description:

Parameter	Type	Description	Value
BrightScreenTimeBean	Class		isOpen, Switch ON YES or OFF NO startHour, start time hour startMin, start time minute endHour, end time hour endMin, end time minute

Example of usage:

```
// Subscribe
DHBleSdk.subscribeData(brightTimeCallback)

// Set
val briScreenTime = BrightScreenTimeBean()
briScreenTime.isOpen = true
briScreenTime.startHour = 20 // Sleep from 8 PM to 8 AM
briScreenTime.startMin = 0
briScreenTime.endHour = 8
briScreenTime.endMin = 0

DHBleSdk.setRingBrightScreenSleepTime(briScreenTime)
```

```
// Get
DHBleSdk.getRingBrightScreenSleepTime()
```

3.2.1.14 Messages and Calls

3.2.1.14.1 Message Push

> APP actively pushes message notifications to the device (not ANCS). The message switch is controlled by the APP itself.
>
> Incoming-call notifications from WeChat and similar apps are not Android system phone states. The APP must extend `NotificationListenerService` to listen for and push these notifications.

Method Description:

`fun setPushMsgJL(msgPushBean: MsgPushBean)`

Parameter Description:

Parameter	Type	Description	Value
MsgPushBean	Class		See MsgPushBean class definition

Example of usage:

```
val messageBean = MsgPushBean()
messageBean.appId = "com.ten.wenxin"
messageBean.title = "1111"
messageBean.content = "8888"
DHBleSdk.setPushMsgJL(messageBean)
```

3.2.1.14.2 Call Notification and Control

> When the APP receives an Android system call, call `controlCallRemindJL()` to push the call information to the device.
>
> `controlPhoneJL()` is used to synchronize the operation after the user answers or hangs up.
>
> When the device triggers an answer or hang-up operation, the APP receives and handles it through `CallRemindCallback`.
>
> The APP must use `TelephonyManager` to listen for Android system phone states. It can register for the `TelephonyManager.ACTION_PHONE_STATE_CHANGED` broadcast and call the corresponding API according to the ringing, answered, and hang-up states.

Method Description:

`fun controlCallRemindJL(callRemindBean: CallRemindBean)`

`fun controlPhoneJL(controlType: Int)`

Parameter Description:

Parameter	Type	Description	Value
callRemindBean	CallRemindBean	Call information	See the class definition
controlType	Int	User operation state	0: Answer 1: Hang up

`CallRemindCallback.onResult()` return values:

Value	Description
1	Device requests answering
2	Device requests hanging up

Example of usage:

```
// Push call information to the device when the APP receives a system call
val callRemindBean = CallRemindBean().apply {
    phoneNum = "13800138000"
    phoneName = "Contact"
}
DHBleSdk.controlCallRemindJL(callRemindBean)

// Synchronize the state after the user answers
DHBleSdk.controlPhoneJL(0)

// Synchronize the state after the user hangs up
DHBleSdk.controlPhoneJL(1)

// Receive answer or hang-up operations from the device
DHBleSdk.subscribeData(object : CallRemindCallback {
    override fun onResult(data: Int?) {
        when (data) {
            1 -> { /* Answer the call */ }
            2 -> { /* Hang up the call */ }
        }
    }

    override fun onFail(errorCode: Int) {}
    override fun onSuccess() {}
})
```

3.2.1.14.3 Music Control

> When the device triggers previous-track or next-track control, the APP should listen for the device command and perform the corresponding action.
>
> Subscribe to `MusicPushSettingCallback` to receive device music control events.

MusicPushSettingCallback return values:

Value	Description
3	Previous track
4	Next track

Example of usage:

```
DHBleSdk.subscribeData(object : MusicPushSettingCallback {
    override fun onResult(data: Int?) {
        when (data) {
            3 -> { /* Previous */ }
            4 -> { /* Next */ }
        }
    }
    override fun onFail(errorCode: Int) {}
    override fun onSuccess() {}
})
```

3.2.1.15 Getting and Setting whether the Reminder Function is Enabled

> Set whether the reminder function is enabled;
>
> Configuration table attribute: `isRememberSwitch`
>
> Subscribe to `MuslimCountSwitchCallback` to get the result.

Method Description:

`fun deviceRememberSwitch(status: Int)`

`fun deviceRememberSwitchGet()`

Parameter Description:

Parameter	Type	Description	Value
status	Int		0: Off 1: On

Example of usage:

```
//set up
DHBleSdk.subscribeData(muslimCountSwitchCallback)
DHBleSdk.deviceRememberSwitch(1)

//Get
DHBleSdk.deviceRememberSwitchGet()
```

Muslim Count Real-time Monitoring

When the user taps the counter on the device, the device actively reports the current count. The push type is `DevicePushType.MUSLIM_COUNT` and `value` is a `MuslimCountItemBean` (count/timeMills). See 3.2.1.21 Device Push Notification for how to subscribe.

3.2.1.16 Getting and Setting Heart Rate/Blood Oxygen Alarm Configuration

> This function sets the heart rate and blood oxygen notification alarm data; alarm notifications will be sent in real-time via `HrBoActualReminderCallback`.
>
> Configuration table attribute: `isSupportHrReminder`
>
> Subscribe to `HrReminderCallback` and `BoReminderCallback` to get the results.

Method Description:

`fun deviceGetHrAlertCmd()`

`fun deviceSetHrAlertCmd(status: Int, value: Int, underValue:Int)`

`fun deviceGetBoAlertCmd()`

`fun deviceSetBoAlertCmd(status: Int, value: Int)`

Parameter Description:

Parameter	Type	Description	Value
status	Int		1: On, 0: Off; overValue: Alarm value, default value is heart rate exceeding 160, blood oxygen below 94%,
value	Int		Exceeding alarm value, default value is heart rate exceeding 160
underValue	Int		Below value alarm; If the retrieved value is 0xff, it means this function is not supported;

Note: If `underValue` obtained through `deviceGetHrAlertCmd()` is 0xff, it means this function is not supported.

Real-time alarm return value HrBoActualReminderBean description:

HrBoActualReminderBean Parameter	Type	Description	Value
type	Int		0: Heart rate exceeding alarm, 1: Blood oxygen alarm; 2: Heart rate below alarm
remindValue	Int		Alarm value

Example of usage:

```
//Setting
DHBleSdk.subscribeData(hrReminderCallback)
DHBleSdk.deviceSetHrAlertCmd(1, 140, 0xff)

//Getting
DHBleSdk.subscribeData(hrReminderCallback)
DHBleSdk.deviceGetHrAlertCmd()


//Alarm result notification push
DHBleSdk.subscribeData(hrBoActualReminderCallback) //Alert Message

private val hrBoActualReminderCallback by lazy {
```

```
  object : HrBoActualReminderCallback {
    override fun onResult(data: HrBoActualReminderBean) {
      Log.e("RWSDK", "output: HrBoActualReminderCallback data " + data.type + " value " +
data.remindValue)
      if (data.type == 0) { // HR Over Value Alarm

      } else if (data.type == 1) { // SP02 Over Value Alarm

      } else if (data.type == 2) { // HR Under Value Alarm

      }
    }

    override fun onFail(errorCode: Int) {
      Log.e("RWSDK", "output: HrBoActualReminderCallback errorCode " + errorCode)
    }

    override fun onSuccess() {
      Log.e("RWSDK", "output: HrBoActualReminderCallback onSuccess")
    }

  }
}
```

3.2.1.17 Get and Set Screen Brightness Duration

> Configuration table attribute: `isBrightScreenTime`
>
> Subscribe to `BrightTimeCallback` to get the result.

Method Description:

`fun getBrightScreenTimeJL()`

`fun setBrightScreenTimeJL(briScreenTime: BrightScreenTimeBean)`

Parameter Description:

BrightScreenTimeBean Parameter	Type	Description	Value
timeSecond	Int	Screen brightness duration, in seconds (s), range 0-30s;	
durationNums	String	Supported duration values by the device, separated by commas if available;	

Example of usage:

```
// Set
val briScreenTime = BrightScreenTimeBean()
briScreenTime.timeSecond = 10 // Screen brightness for 10s
DHBleSdk.subscribeData(brightTimeCallback)
DHBleSdk.setBrightScreenTimeJL(briScreenTime)

// Get
DHBleSdk.getBrightScreenTimeJL()
```

3.2.1.18 Getting and Setting Wrist-Raise Screen Activation Time

> Configuration table attribute: `isRaiseBrightScreen`
>
> Subscribe to `BrightCallback` to get the result.

Method Description:

`fun getRaiseBrightScreenJL()`

`fun setRaiseBrightScreenJL(brightScreenBean: BrightScreenBean)`

Parameter Description:

BrightScreenBean Parameter	Type	Description	Value
isOpen	Int	true: enabled; false: disabled	
startHour	Int	Start time hour	
startMin	Int	Start time minute	
endHour	Int	End time hour	
endMin	Int	End time minute	

Example of usage:

```
// Setting
val rasieScreenTime = BrightScreenBean()
rasieScreenTime.isOpen = true
rasieScreenTime.startHour = 8 // Wrist-raise screen activation from 8 AM to 8 PM
rasieScreenTime.startMin = 0
rasieScreenTime.endHour = 20
rasieScreenTime.endMin = 0
DHBleSdk.subscribeData(raiseBrightTimeCallback)
DHBleSdk.setRaiseBrightScreenJL(rasieScreenTime)

// Getting
DHBleSdk.getRaiseBrightScreenJL()
```

3.2.1.19 Set Time Format (12-Hour / 24-Hour)

> This setting only applies to devices with a display.
>
> Subscribe to `CommonStatusCallback` to get the result.

Method Description:

`fun ringSetTimeformat(type: Int)`

Parameter Description:

Parameter	Type	Description	
timeformat	Int	0: 24-hour 1: 12-hour	

Example of usage:

```
DHBleSdk.subscribeStatus(object : CommonStatusCallback{
  override fun onSuccess(id: Int) {
    Log.e("RWSDK", "time format set ok")
  }

  override fun onFail(id: Int, errorCode: Int) {
    Log.e("RWSDK", "time format set failed")
  }
})
DHBleSdk.ringSetTimeformat(0) //24-hour
```

3.2.1.20 Alarm Vibration Duration Setting and Getting

> Set the alarm vibration count;
>
> Configuration table property: `isSupportAlarmVibrationDuration`
>
> Subscribe to `AlarmVibrationDurationCallback` to get the result.

Method Description:

`fun setAlarmVibrationDuration(count: Int)`

`fun getAlarmVibrationDuration()`

Parameter Description:

Parameter	Type	Description	Value
count	Int	Integer	Vibration count (0-6), default 2, 0 means no vibration

Example of usage:

```
//Set
DHBleSdk.subscribeData(alarmVibrationDurationCallback)
DHBleSdk.setAlarmVibrationDuration(2) //2 times

//Get
DHBleSdk.subscribeData(alarmVibrationDurationCallback)
DHBleSdk.getAlarmVibrationDuration()
```

3.2.1.21 Device Push Notification

> The unified channel for device-initiated reports: device events such as battery push, recording status push, Muslim count reporting and touch events are all dispatched through `OnDevicePushListener`, distinguished by `PushData.type`.
>
> The callback runs synchronously on the BLE thread. Do not perform time-consuming work; switch to the main thread before updating the UI.

PushData structure:

Field	Type	Description
type	DevicePushType	Push type, see the type table below
value	Any	Parsed business object of the corresponding type
timestamp	Long	Unix milliseconds when the SDK received the push

DevicePushType description:

DevicePushType	Data	value entity	See
POWER	Battery and charging status	PowerBean	3.2.1.4.1 Real-time Battery Monitoring
RECORD_STATUS	Recording status push	RecordStatusBean	3.2.5.2 Query Recording Status
RECORD_REALTIME_DATA	Real-time recording audio	RecordRealtimeDataBean	3.2.5.7 Real-time Recording
COUNT_CONTROL	Counting recognition state	Boolean: true enabled, false disabled	3.2.1.33 Counting Recognition Control and State Queries
MUSLIM_COUNT	Muslim count real-time report	MuslimCountItemBean	3.2.1.15 Reminder Function
TOUCH_EVENT	Touch / fall / SOS / repeated taps	TouchEventBean	Requires firmware support. keyType: 1: touch key, 2: fall (requires enabling 3.2.1.24), 3: SOS, 4: repeated taps. touchType: for touch keys, 1: single tap, 2: double tap, 3: triple tap, 4: long press, 5: shake, 6: Release after a long press; defaults to 1 for fall; unused for SOS; contains the tap count (1-9) for repeated taps

Method Description:

`fun addOnDevicePushListener(listener: OnDevicePushListener)`

`fun removeOnDevicePushListener(listener: OnDevicePushListener)`

Example of usage (one listener dispatching by type):

```
private val devicePushListener = object : OnDevicePushListener {
  override fun onPush(data: PushData) {
    when (data.type) {
      DevicePushType.POWER -> {
        val powerBean = data.value as? PowerBean ?: return
        Log.e("RWSDK", "power=${powerBean.power} charging=${powerBean.powerStatus}")
      }
      DevicePushType.MUSLIM_COUNT -> {
        val item = data.value as? MuslimCountItemBean ?: return
        Log.e("RWSDK", "muslim count=${item.count}")
      }
      DevicePushType.TOUCH_EVENT -> {
        val event = data.value as? TouchEventBean ?: return
        when {
          event.keyType == 3 -> Log.e("RWSDK", "SOS")
          event.keyType == 4 -> Log.e("RWSDK", "Repeated tap count=${event.touchType}")
          event.keyType == 1 && event.touchType == 6 -> Log.e("RWSDK", "Release")
          else -> Log.e("RWSDK", "touch keyType=${event.keyType} touchType=${event.touchType}")
        }
      }
      DevicePushType.RECORD_STATUS -> {
        val recordStatus = data.value as? RecordStatusBean ?: return
        Log.e("RWSDK", "record recording=${recordStatus.isRecording}")
      }
      else -> Unit
    }
  }
}

// Start listening (application-level registration, add/remove per consumer lifecycle).
DHBleSdk.addOnDevicePushListener(devicePushListener)

// Remove the same listener instance when it is no longer needed.
DHBleSdk.removeOnDevicePushListener(devicePushListener)
```

3.2.1.22 Vibration Interval Setting and Getting

> Set the interval time between each vibration, used to adjust vibration rhythm;
>
> Configuration table property: `isSupportVibrationInterval`
>
> Subscribe to `VibrationIntervalCallback` to get the result.

Method Description:

`fun setVibrationInterval(intervalMs: Int)`

`fun getVibrationInterval()`

Parameter Description:

Parameter	Type	Description	Value
intervalMs	Int	Integer	Interval duration (100-1000ms), default 500ms

Example of usage:

```
//Set
DHBleSdk.subscribeData(vibrationIntervalCallback)
DHBleSdk.setVibrationInterval(500) //500ms

//Get
DHBleSdk.subscribeData(vibrationIntervalCallback)
DHBleSdk.getVibrationInterval()
```

3.2.1.23 HR Calibration (Factory Test)

> Start device heart rate calibration mode. After sending the calibration command, the device returns 2 responses:
>
> 1st response: result=0 (calibrating); 2nd response: result≠0 (calibration done).
>
> Subscribe to `FactoryTestCallback` to get results, onResult returns long[]: [0]=testMode, [1]=result.

Method Description:

`fun startFactoryTest(testMode: Int)`

Parameter Description:

Parameter	Type	Description	Value
testMode	Int	Test mode	0x15: HR Calibration

Example of usage:

```
DHBleSdk.subscribeData(factoryTestCallback)
DHBleSdk.startFactoryTest(0x15)

private val factoryTestCallback by lazy {
    object : FactoryTestCallback {
        override fun onResult(data: LongArray?) {
            data?.let {
                if (it[1] == 0L) {
                    Log.e("RWSDK", "HR calibrating...")
                } else {
                    Log.e("RWSDK", "HR calibration done, result=${it[1]}")
                }
            }
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}
```

3.2.1.24 Fall Detection Setting

> Set or get the fall detection switch. When enabled, the device reports through the touch event notification when a fall is detected.
>
> Fall events are received via the `TOUCH_EVENT` type of 3.2.1.21 Device Push Notification; keyType=2 indicates a fall event.
>
> Configuration table attribute: `isSupportFallDetect`
>
> The result is returned via [callback].

Method Description:

```
fun setFallDetect(enable: Boolean, callback: FallDetectCallback)
```

```
fun getFallDetect(callback: FallDetectCallback)
```

Parameter Description:

Parameter	Type	Description	Value
enable	Boolean	Switch	true: on, false: off
callback	FallDetectCallback	Result callback	Set: onSuccess()/onFail(); Query: onResult(Int, 0=off 1=on);

Example of usage:

```
//Query the fall detection switch
DHBleSdk.getFallDetect(object : FallDetectCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "fall detect query failed: $errorCode")
    }
    override fun onResult(data: Int) {
        Log.e("RWSDK", "FallDetect state: $data (0=off, 1=on)")
    }
})

//Enable fall detection
DHBleSdk.setFallDetect(true, object : FallDetectCallback {
    override fun onSuccess() {
        Log.e("RWSDK", "FallDetect enabled")
    }
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "FallDetect set failed: $errorCode")
    }
    override fun onResult(data: Int) = Unit
})
```

3.2.1.25 Count Reminder Interval Setting

> Set or get the count reminder interval. When enabled, after the user completes a count operation, the device starts timing and vibrates once when the interval is reached to remind the user to continue counting.
>
> Configuration table property: `isSupportCountReminder`

> Subscribe to `CountReminderIntervalCallback` to get the result.

Method Description:

`fun setCountReminderInterval(intervalMinutes: Int)`

`fun getCountReminderInterval()`

Parameter Description:

Parameter	Type	Description	Value
intervalMinutes	Int	Interval minutes	0: off, 30/60/90/120: reminder interval

Example of usage:

```
//Get count reminder interval
DHBleSdk.subscribeData(countReminderCallback)
DHBleSdk.getCountReminderInterval()

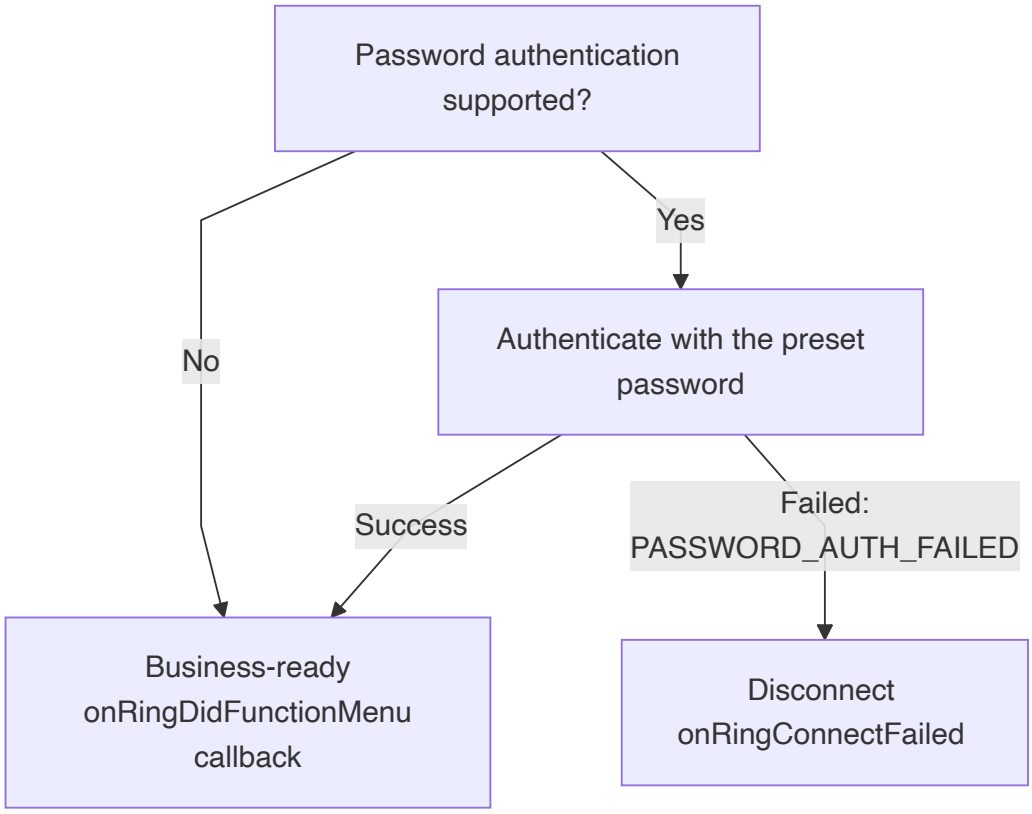
//Set count reminder interval to 60 minutes
DHBleSdk.subscribeData(countReminderCallback)
DHBleSdk.setCountReminderInterval(60)

//Turn off count reminder
DHBleSdk.subscribeData(countReminderCallback)
DHBleSdk.setCountReminderInterval(0)

private val countReminderCallback by lazy {
    object : CountReminderIntervalCallback {
        override fun onResult(data: Int?) {
            Log.e("RWSDK", "CountReminderInterval: $data min (0=off)")
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {}
    }
}
```

3.2.1.26 Device Password Authentication

> Check `isSupportDevicePasswordAuth` in the device configuration table to determine whether the device supports password authentication.
>
> The password must contain four digits. A `null` or empty value is treated as the default password `0000`.
>
> For a supported device, `onRingDidFunctionMenu` is called only after authentication succeeds. If authentication fails, the SDK disconnects and returns `RingBleError.PASSWORD_AUTH_FAILED`. Unsupported devices continue to use the original connection flow.



3.2.1.26.1 Set the Automatic Authentication Password

`fun prepareAutoPassword(password: String?)`

> Set the password used by the SDK for automatic authentication. It may be configured after SDK initialization, but it must be called before connecting.

Input Parameter:

Parameter	Type	Description
`password`	`String`	Four-digit password; `null` or an empty string is treated as `0000`

Callback Result:

Callback Method	Result	Description
`onRingDidFunctionMenu`	`SupportMenuBean`	Authentication succeeded; the device is business-ready
`onRingConnectFailed`	`RingBleError.PASSWORD_AUTH_FAILED`	Authentication failed; the SDK disconnects the device

Example:

```
DHBleSdk.setConnectBleCallback(this)
DHBleSdk.prepareAutoPassword("1234")
DHBleSdk.connectDeviceWithModel(bleDevice)

override fun onRingDidFunctionMenu(device: BleDevice?, supportMenuBean: SupportMenuBean) {
    Log.e("RWSDK", "Device ready")
}

override fun onRingConnectFailed(device: BleDevice?, reason: RingBleError) {
    if (reason == RingBleError.PASSWORD_AUTH_FAILED) {
        Log.e("RWSDK", "Device password authentication failed")
    }
}
```

3.2.1.26.2 Modify the Device Password

`fun modifyDevicePwd(password: String?, callback: CustomStatusCallback)`

> Modify the device password after the device is connected and authenticated. For a normal unbind operation, first change the password to `0000`; only clear the local binding and disconnect after the success callback.

Example:

```
DHBleSdk.modifyDevicePwd("0000", object : CustomStatusCallback {
    override fun onSuccess() {
        //The password is now 0000. Continue local unbinding and disconnect.
        DHBleSdk.disconnect()
    }

    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "Modify password failed: $errorCode")
    }
})
```

3.2.1.26.3 Prepare an Authorized Password Reset

`fun preparePasswordReset(targetPassword: String?)`

> If the original device password is unavailable, the app may call this method after confirming that the user is authorized to reset the device. It sets a new target password for the next connection so that a password-protected device does not become permanently unusable. The app decides how reset authorization is verified; scanning a QR code on the package is only one possible method.

Parameter:

Parameter	Type	Description
`targetPassword`	`String`	New four-digit password; `null` or an empty string is treated as `0000`

Example:

```
// Call after the app confirms that the user is authorized to reset the device.
DHBleSdk.preparePasswordReset("5678")
DHBleSdk.connectDeviceWithModel(bleDevice)
```

3.2.1.26.4 Device Identity Authentication

`fun deviceChallenge(challengeHex: String, callback: DeviceChallengeCallback)`

> Configuration table attribute: `isSupportDeviceChallenge`. Only available on devices supporting this capability.
>
> Passes the cloud-generated challenge through to the device; the device computes HMAC-SHA256 with the factory-preset key and returns the full response. The SDK only sends the challenge and returns the device response; **business verification of the HMAC result is handled by the app and the cloud**.

Parameter Description:

Parameter	Type	Description	Value
challengeHex	String	Random challenge generated by the cloud, hexadecimal string	64 characters (=32 bytes), spaces/colons/dashes allowed; invalid format or length fails locally with `onFail`
callback	DeviceChallengeCallback	Result callback	The result is returned via onResult(data: String), the response as a hexadecimal string

Example of usage:

```
val challengeCallback = object : DeviceChallengeCallback {
  override fun onResult(data: String) {
    //hex string of the 32-byte HMAC-SHA256 response, forward to the cloud for verification
    Log.e("RWSDK", "device challenge response=$data")
  }

  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "device challenge failed: $errorCode")
  }
}

//The challenge is generated by the cloud (random bytes as a hex stand-in here)
val challengeHex = CmdHelper.bytesToHex(ByteArray(32).also { java.util.Random().nextBytes(it) })
DHBleSdk.deviceChallenge(challengeHex, challengeCallback)
```

3.2.1.27 Instant Screen Control

> Configuration-table property: `isSupportScreenControl`. Use this feature only when the device reports support.

Methods:

`fun setScreenOn(isOn: Boolean)`

Parameter:

Parameter	Type	Description
isOn	Boolean	`true`: turn the screen on; `false`: turn the screen off

Callbacks:

- Setting results are returned through `ScreenStatusCallback.onSuccess/onFail`.

Example:

```
// Set the current screen state.
val setScreenCallback = object : ScreenStatusCallback {
  override fun onResult(data: Boolean?) {
  }

  override fun onSuccess() {
    Log.e("RWSDK", "screen control success")
    DHBleSdk.dispose(this)
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "screen control failed: $errorCode")
    DHBleSdk.dispose(this)
  }
}
DHBleSdk.subscribeData(setScreenCallback)
DHBleSdk.setScreenOn(true)
```

3.2.1.28 Metric/Imperial Unit Settings and Retrieval

> Configuration-table property: `isSupportUnitSetting`. Use this feature only when the device reports support.
>
> Subscribe to `UnitSettingCallback` to receive operation results.

Methods:

`fun setMeasureUnit(type: Int)`

`fun getMeasureUnit()`

Parameter:

Parameter	Type	Description
type	Int	`0`: metric; `1`: imperial

Callbacks:

- A successful setting operation is returned through `onSuccess()`, and a failure through `onFail(errorCode)`.
- The current unit is returned through `onResult(data)` when querying.

Setting example:

```
val unitSettingCallback = object : UnitSettingCallback {
  override fun onResult(data: Int?) {
  }

  override fun onSuccess() {
    Log.e("RWSDK", "measure unit set success")
    DHBleSdk.dispose(this)
  }

  override fun onFail(errorCode: Int) {
```

```
    Log.e("RWSDK", "measure unit set failed: $errorCode")
    DHBleSdk.dispose(this)
  }
}

DHBleSdk.subscribeData(unitSettingCallback)
DHBleSdk.setMeasureUnit(0)
```

Retrieval example:

```
val unitSettingCallback = object : UnitSettingCallback {
  override fun onResult(data: Int?) {
    Log.e("RWSDK", "measure unit=$data")
    DHBleSdk.dispose(this)
  }

  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "measure unit get failed: $errorCode")
    DHBleSdk.dispose(this)
  }
}

DHBleSdk.subscribeData(unitSettingCallback)
DHBleSdk.getMeasureUnit()
```

3.2.1.29 Sedentary Reminder Setting and Getting

> Configuration table attribute: `isSupportSedentary`. Only available on devices supporting this capability.
>
> Within the specified period, the device vibrates to remind the user when continuous sedentary time exceeds the reminder interval.

Method Description:

`fun setSedentaryRemind(reminderBean: DrinkReminderBean, callback: ReminderSettingCallback)`

`fun getSedentaryRemind(callback: ReminderSettingCallback)`

> The set result is returned via `onSuccess()` / `onFail()`, and the query result via `onResult(DrinkReminderBean)`.

DrinkReminderBean parameters:

Parameter	Type	Description	Value
isOpen	Boolean	Sedentary reminder switch	true: on; false: off
startHour / startMin	Int	Reminder period start	0-23 / 0-59
endHour / endMin	Int	Reminder period end	0-23 / 0-59
remindDuration	Int	Reminder interval, in minutes	Time of continuous sedentary time before a reminder triggers

Example of usage:

```
//Query current configuration
DHBleSdk.getSedentaryRemind(object : ReminderSettingCallback {
  override fun onResult(data: DrinkReminderBean?) {
    Log.e("RWSDK", "sedentary open=${data?.isOpen} " +
        "${data?.startHour}:${data?.startMin}-${data?.endHour}:${data?.endMin} " +
        "interval=${data?.remindDuration}min")
  }

  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "sedentary query failed: $errorCode")
  }
})

//Set: on, 09:00-18:00, 60-minute interval
DHBleSdk.setSedentaryRemind(DrinkReminderBean().apply {
  isOpen = true
  startHour = 9
  startMin = 0
  endHour = 18
  endMin = 0
  remindDuration = 60
}, object : ReminderSettingCallback {
  override fun onResult(data: DrinkReminderBean?) {
  }

  override fun onSuccess() {
    Log.e("RWSDK", "sedentary set success")
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "sedentary set failed: $errorCode")
  }
})
```

3.2.1.30 Drinking Reminder Setting and Getting

> Configuration table attribute: `isDrink`. Only available on devices supporting this capability.
>
> Within the specified period, the device vibrates periodically at the reminder interval to remind the user to drink water.

Method Description:

`fun setDrinkRemind(reminderBean: DrinkReminderBean, callback: ReminderSettingCallback)`

`fun getDrinkRemind(callback: ReminderSettingCallback)`

> The set result is returned via `onSuccess()` / `onFail()`, and the query result via `onResult(DrinkReminderBean)`.

DrinkReminderBean parameters:

Parameter	Type	Description	Value
isOpen	Boolean	Drinking reminder switch	true: on; false: off
startHour / startMin	Int	Reminder period start	0-23 / 0-59
endHour / endMin	Int	Reminder period end	0-23 / 0-59
remindDuration	Int	Reminder interval, in minutes	Trigger interval of drinking reminders within the period

Example of usage:

```
//Query current configuration
DHBleSdk.getDrinkRemind(object : ReminderSettingCallback {
  override fun onResult(data: DrinkReminderBean?) {
    Log.e("RWSDK", "drink open=${data?.isOpen} " +
        "${data?.startHour}:${data?.startMin}-${data?.endHour}:${data?.endMin} " +
        "interval=${data?.remindDuration}min")
  }

  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "drink query failed: $errorCode")
  }
})

//Set: on, 08:00-22:00, 30-minute interval
DHBleSdk.setDrinkRemind(DrinkReminderBean().apply {
  isOpen = true
  startHour = 8
  startMin = 0
  endHour = 22
  endMin = 0
  remindDuration = 30
}, object : ReminderSettingCallback {
  override fun onResult(data: DrinkReminderBean?) {
  }
```

```
  override fun onSuccess() {
    Log.e("RWSDK", "drink set success")
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "drink set failed: $errorCode")
  }
})
```

3.2.1.31 Custom vibration control

Requires `SupportMenuBean.isSupportVibrationControl`.

```
fun controlVibration(mode: Int, strength: Int, groupCount: Int,
                     frequency: Int, pause: Int, callback: CustomStatusCallback)
```

Parameter	Description
mode	1–15: vibrate N times; 255: continuous or repeating groups
strength	0: stop; 1: weak; 2: medium; 3: strong
groupCount	Pulses per group, 1–15; 0 when not using grouped repetition
frequency	Pulses per second, 1–10; 0 when not using grouped repetition
pause	Pause between groups, 1–255 in 100 ms units; 0 otherwise
callback	onSuccess() / onFail(errorCode); automatically released, no subscribe/dispose

Use zero for all three group parameters in single-round or continuous mode. Grouped repetition requires mode=255 and all three group parameters. Set strength=0 to stop (group parameters are ignored). Invalid parameters throw IllegalArgumentException. Wait for the previous result before issuing another request.

```
val callback = object : CustomStatusCallback {
    override fun onSuccess() {}
    override fun onFail(errorCode: Int) {}
}
DHBleSdk.controlVibration(3, 2, 0, 0, 0, callback) // Three medium pulses
// Other alternatives (call separately):
// controlVibration(255, 2, 0, 0, 0, callback) // Continuous
// controlVibration(255, 2, 3, 2, 10, callback) // Repeating groups, 1 s pause
// controlVibration(255, 0, 0, 0, 0, callback) // Stop
```

3.2.1.32 Custom Number Display

> Displays an APP-specified number on the device for 3 seconds. Calling the method again updates the number and restarts the 3-second display period. The APP defines the meaning and interpretation of the number.
>
> Function configuration property: `isSupportCustomNumberDisplay`. Only devices supporting this capability can use this feature.

Method Description:

`fun showCustomNumber(number: Int, callback: CustomStatusCallback)`

Parameter Description:

Parameter	Type	Description
number	Int	Number to display, 0–9999, up to 4 digits. Values below 0 are treated as 0; values above 9999 are treated as 9999.
callback	CustomStatusCallback	`onSuccess()`: operation succeeded; `onFail(errorCode)`: operation failed. Automatically released after the result; no subscription or `dispose` is required.

Example of usage:

```
DHBleSdk.showCustomNumber(627, object : CustomStatusCallback {
  override fun onSuccess() {
    Log.e("RWSDK", "custom number display success")
  }

  override fun onFail(errorCode: Int) {
    Log.e("RWSDK", "custom number display failed, errorCode=$errorCode")
  }
})
```

3.2.1.33 Counting Recognition Control and State Queries

> Requires `isSupportCountControl`. Enabling or disabling recognition does not clear the accumulated count.

```
fun setCountControl(armed: Boolean, callback: CustomStatusCallback)
fun getCountControl(callback: CountControlCallback)
```

Parameter / callback	Description
armed	true: enabled (Arm); false: disabled (Disarm)
CustomStatusCallback	`onSuccess()` on successful settings; `onFail(errorCode)` on failure
CountControlCallback	`onResult(Boolean)` returns the current state; `onFail(errorCode)` on failure

Wait for the previous operation's callback before issuing another set or query.

Setting example:

```
DHBleSdk.setCountControl(true, object : CustomStatusCallback {
    override fun onSuccess() { Log.e("RWSDK", "counting enabled") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "count control failed=$errorCode") }
})
```

Query example (call separately from the setting operation):

```
DHBleSdk.getCountControl(object : CountControlCallback {
    override fun onSuccess() = Unit
    override fun onResult(data: Boolean) { Log.e("RWSDK", "counting enabled=$data") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "get count control failed=$errorCode") }
})
```

State changes arrive as `DevicePushType.COUNT_CONTROL`, with a Boolean `value`. Real-time counts arrive as `DevicePushType.MUSLIM_COUNT`, with a `MuslimCountItemBean`. See 3.2.1.21 for listener registration.

3.2.1.34 Set and Get Customer Model

> Requires `isSupportCustomerModel`. The customer model defaults to the device model and can distinguish products in your application. It does not change `deviceClazz`, which must still be used to match OTA firmware.

```
fun setCustomerModel(model: String, callback: CustomStatusCallback)
fun getCustomerModel(callback: CustomerModelCallback)
```

Parameter/Callback	Description
model	Customer model, up to 8 characters; English letters and digits only
CustomStatusCallback	`onSuccess()` when saved; `onFail(errorCode)` on failure
CustomerModelCallback	`onResult(String)` returns the current customer model; `onFail(errorCode)` on failure

```
DHBleSdk.setCustomerModel("MODEL001", object : CustomStatusCallback {
    override fun onSuccess() { Log.e("RWSDK", "customer model saved") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "set failed=$errorCode") }
})

DHBleSdk.getCustomerModel(object : CustomerModelCallback {
    override fun onSuccess() = Unit
    override fun onResult(data: String) { Log.e("RWSDK", "customer model=$data") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "get failed=$errorCode") }
})
```

3.2.1.35 Custom Text Display with Duration

> Requires `isSupportCustomTextDisplay`. Display English letters, digits, or a combination, for the specified duration. Calling again updates the content and restarts the display timer.

`fun showCustomText(text: String, durationSeconds: Int, callback: CustomStatusCallback)`

Parameter	Type	Description
text	String	Up to five English letters or digits; longer content is truncated to the first five characters. Must not be empty.
durationSeconds	Int	Display duration in seconds; values below 1 are treated as 1 and values above 255 as 255.
callback	CustomStatusCallback	`onSuccess()` is called when the device accepts the command, without waiting for the display to finish; `onFail(errorCode)` reports failure.

Invalid content is reported through `onFail(Constants.ERROR_INFO)`. Wait for the previous display request to complete before sending another.

```
if (supportMenu.isSupportCustomTextDisplay) {
    DHBleSdk.showCustomText("A627", 5, object : CustomStatusCallback {
        override fun onSuccess() {
            Log.e("RWSDK", "Custom text display succeeded")
        }
        override fun onFail(errorCode: Int) {
            Log.e("RWSDK", "Custom text display failed: $errorCode")
        }
    })
}
```

3.2.2 Health Data Synchronization (Real-time Single Measurement and All-day Monitoring)

> There are two ways to detect health data: real-time single measurement and all-day monitoring. Health data includes heart rate, blood oxygen, stress, HRV, sleep, etc. **Sleep data does not have real-time measurement.**
>
> (1) Real-time single measurement: The APP starts the device to perform a single measurement, and the result is returned immediately after the measurement is completed.
>
> (2) All-day monitoring: The interval time can be set, for example, 30 minutes or 60 minutes, and the device will perform the measurement and save the value; **if the app does not synchronize continuously, the device can save 3-6 days of data.**

3.2.2.0 Real-time Monitoring - Individual Measurement

> **Note**
>
> Start or stop a single measurement of heart rate, blood oxygen, HRV, stress, blood glucose, blood pressure, or temperature. Individual and composite measurements cannot run at the same time; start the next measurement only after the current one finishes or is stopped manually.

Method Description:

```
fun controlOpen(type: Int, dataType: Int, callback: HealthMeasurementCallback)
```

Parameter Description:

Parameter	Type	Description	Value
type	Int	Start/Stop	1: start / 0: stop
dataType	Int	Data type	`HealthDataType.code`, see the type table below
callback	HealthMeasurementCallback	Result callback	onStarted: start confirmed; onData(HealthRealtimeValue): real-time value during measurement (dataType/timestamp/value/extraValue); onFinished(HealthMeasurementResult): measurement finished (result.isSuccess) or failed/timed out (result.errorCode)

HealthDataType description:

HealthDataType	Item
HEART_RATE	Heart rate
BLOOD_PRESSURE	Blood pressure
TEMPERATURE	Temperature
BLOOD_OXYGEN	Blood oxygen
HRV	Heart rate variability
STRESS	Stress
BLOOD_SUGAR	Blood glucose

Results: `onStarted()` confirms that measurement has started, `onData()` delivers real-time values, and `onFinished()` reports the session outcome. Receiving a value does not mean that the measurement has finished.

In `onFinished(result)`, `result.isSuccess=true` indicates normal completion. If the device is not worn, `isSuccess=false` and `result.errorCode=Constants.ERROR_NOT_WORN`; put the device on and try again. Other failures or timeouts are identified by `result.errorCode`.

Example of usage:

```
//Start heart rate measurement
DHBleSdk.controlOpen(1, HealthDataType.HEART_RATE.code, object : HealthMeasurementCallback {
    override fun onStarted() {
        Log.e("RWSDK", "heart rate measurement started")
    }

    override fun onData(data: HealthRealtimeValue) {
        Log.e("RWSDK", "real-time heart rate: ${data.value}")
    }

    override fun onFinished(result: HealthMeasurementResult) {
        if (result.isSuccess) {
            Log.e("RWSDK", "measurement finished")
        } else if (result.errorCode == Constants.ERROR_NOT_WORN) {
            Log.e("RWSDK", "Ring not worn. Please put it on and try again.")
        } else {
            Log.e("RWSDK", "measurement failed/timeout: ${result.errorCode}")
        }
    }
})
```

To stop early, call separately:

```
DHBleSdk.controlOpen(0, HealthDataType.HEART_RATE.code, object : HealthMeasurementCallback {
    override fun onStarted() = Unit
    override fun onData(data: HealthRealtimeValue) = Unit
    override fun onFinished(result: HealthMeasurementResult) = Unit
})
```

3.2.2.1 Real-time Monitoring - Composite Measurement

> Requires `isSupportCompositeMeasurement`. Only items actually reported by the device are returned. Do not run individual and composite measurements at the same time.

Method:

`fun controlCompositeMeasurement(type: Int, callback: CompositeMeasurementCallback)`

Parameters and callbacks:

Parameter / callback	Description
type	`1`: start; `0`: stop
onStarted()	The device has confirmed that measurement has started
onData(results)	`List<HealthRealtimeValue>` containing all items from one complete report; each item's `dataType` identifies the measurement, and all items share the same `timestamp`
onFinished(result)	The session has ended or failed: `result.isSuccess=true` indicates normal completion; `result.errorCode=Constants.ERROR_NOT_WORN` means the device is not worn—put it on and try again; other error codes indicate failure. `result.dataType` is `HealthDataType.COMPOSITE.code`

Result fields: each item in `results` is a `HealthRealtimeValue`.

Field	Type	Description
dataType	Int	Measurement type, identified by `HealthDataType.code` in the individual measurement parameter table in 3.2.2.0
timestamp	Long	Unix timestamp in seconds; all items in one report share the same timestamp
value	Double	Measured value; systolic pressure for blood pressure
extraValue	Double	Diastolic pressure for blood pressure; 0 for other items

Receiving data does not indicate that the session has finished; wait for `onFinished()`.

Example:

```
if (supportMenu.isSupportCompositeMeasurement) {
    DHBleSdk.controlCompositeMeasurement(1, object : CompositeMeasurementCallback {
        override fun onStarted() = Unit
        override fun onData(results: List<HealthRealtimeValue>) {
            // A complete report; use each item's dataType to update the corresponding value.
            results.forEach { item ->
                Log.e("RWSDK", "type=${item.dataType}, time=${item.timestamp}, value=${item.value},
extra=${item.extraValue}")
            }
        }
        override fun onFinished(result: HealthMeasurementResult) {
            if (result.errorCode == Constants.ERROR_NOT_WORN) {
                Log.e("RWSDK", "Ring not worn. Please put it on and try again.")
            } else {
```

```
                Log.e("RWSDK", "Composite measurement ended: success=${result.isSuccess},
error=${result.errorCode}")
            }
        }
    })
}
```

To stop, call separately:

```
DHBleSdk.controlCompositeMeasurement(0, object : CompositeMeasurementCallback {
    override fun onStarted() = Unit
    override fun onData(results: List<HealthRealtimeValue>) = Unit
    override fun onFinished(result: HealthMeasurementResult) = Unit
})
```

3.2.2.2 Continuous monitoring - Set the interval for continuous monitoring of health data.

> **Note**
>
> The monitoring period is fixed to all day (00:00–23:59). See the table below for each type's required capability and recommended intervals. These intervals are recommended values; other intervals require device support. Shorter intervals increase power consumption and shorten battery life between charges.

> All continuous monitoring items (heart rate/blood oxygen/HRV/pressure/blood glucose/blood pressure/temperature) are distinguished by a single `HealthMonitorType` parameter;
>
> Callback: `ReminderSettingCallback`; Entity: `HealthMonitorBean`.

Method Description:

`fun setHealthMonitor(type: HealthMonitorType, monitorBean: HealthMonitorBean, callback: ReminderSettingCallback)`

`fun getHealthMonitor(type: HealthMonitorType, callback: ReminderSettingCallback)`

Type description:

HealthMonitorType	Item	Recommended interval	Configuration table attribute
HEART_RATE	Heart rate	30/60 minutes	isHr
BLOOD_OXYGEN	Blood oxygen	60 minutes	isBloodOxy
HRV	HRV	60 minutes	isHrv
STRESS	Pressure	60 minutes	isPressure
BLOOD_SUGAR	Blood glucose	60 minutes	isBloodSugar
BLOOD_PRESSURE	Blood pressure	60 minutes	isBloodPress
BODY_TEMPERATURE	Temperature	30/60 minutes	isSupportTemperatureMonitoring

Parameter Description:

Parameter	Type	Description
type	HealthMonitorType	Enum, monitoring type, see the type table above
monitorBean	HealthMonitorBean	Monitoring configuration, fields listed below
callback	ReminderSettingCallback	Set result: onSuccess()/onFail(); Query result: onResult(DrinkReminderBean). The callback is passed directly as a method parameter; no extra subscription or `dispose()` call is needed.

HealthMonitorBean fields:

Field	Type	Description
isOpen	boolean	true on / false off
remindDuration	int	Monitoring interval in minutes; see the type table for recommended values
startHour	int	Fixed 0, cannot be modified
startMin	int	Fixed 0, cannot be modified
endHour	int	Fixed 23, cannot be modified
endMin	int	Fixed 59, cannot be modified

Example of usage:

```
//Unified entry: set heart rate monitoring
val monitorBean = HealthMonitorBean()
monitorBean.isOpen = true
monitorBean.remindDuration = 30 // Recommended monitoring interval: 30 minutes
monitorBean.startHour = 0 //fixed
monitorBean.startMin = 0 //fixed
monitorBean.endHour = 23 //fixed
monitorBean.endMin = 59  //fixed
DHBleSdk.setHealthMonitor(HealthMonitorType.HEART_RATE, monitorBean, object : ReminderSettingCallback
{
    override fun onSuccess() { /* set success */ }
    override fun onFail(errorCode: Int) { /* set failed */ }
    override fun onResult(data: DrinkReminderBean?) { /* query result (with getHealthMonitor) */ }
})

//Unified entry: query blood oxygen monitoring configuration
DHBleSdk.getHealthMonitor(HealthMonitorType.BLOOD_OXYGEN, object : ReminderSettingCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) { /* query failed */ }
    override fun onResult(data: DrinkReminderBean?) { /* current configuration */ }
})
```

3.2.2.3 All-day Monitoring - Synchronize Health History Data

> Synchronizing all health history data will **automatically retrieve health data of supported types according to the configuration table**; calling `syncAllHealthData` will retrieve health data sequentially and provide the results through the `HealthDataSyncCallback` callback.

> `fun removeHealthDataCallBack(syncCallback: HealthDataSyncCallback)` removes the callback;

Interface Description:

`DHBleSdk.syncAllHealthData(this)`

Example of usage:

```
DHBleSdk.syncAllHealthData(this)

public interface HealthDataSyncCallback {
  void onSyncProgress(int var1); // Sync progress

  void onSyncFinish(); // Sync complete

  void onSyncError(int var1);

  void onSyncStep(List<StepSyncBean> var1);

  void onSyncSleep(List<SleepSyncBean> var1);

  void onSyncHr(List<HeartRateSyncBean> var1); // Heart rate

  void onSyncBp(List<BloodPressSyncBean> var1);

  void onSyncBo(List<BloodOxySyncBean> var1); // Blood oxygen

  void onSyncTemp(List<BodyTempSyncBean> var1);

  void onSyncPressure(List<PressureSyncBean> var1); // Pressure

  void onSyncBloodSugar(List<BloodSugarSyncBean> var1); // Blood sugar

  void onSyncHrv(List<HrvSyncBean> var1); // HRV

  void onSyncMuslimCount(List<MuslimCountSyncBean> var1); // Dhikr count
}
```

3.2.2.3.1 Obtaining Single Health Data Content

> **Caution**
>
> The device must support the corresponding health data type; unsupported types will not be synchronized;

Interface Description:

`fun syncHealthDataByType(type: Int, syncCallback: HealthDataSyncCallback)`

Parameter Description:

Parameter	Type	Description	Value
type	Int	Integer	See the definitions in `RingHealthType`;

Example of usage:

```
// Get today's step data
DHBleSdk.syncHealthDataByType(Constants.RingHealthType.TODAY_STEP, this)
```

3.2.2.4 All-Day Monitoring - Health Data Description

`HealthDataSyncCallback` returns a date-based list for each health data type. Each date object provides that day's measurement details through `items`.

> **Important**
>
> In this section, `time`, `timeMills`, `timestamp`, `asleepTime`, and `awakeTime` are Unix timestamps in seconds. `timeMills` is a legacy field name and does not represent milliseconds. Multiply it by `1000` when converting it to a Java timestamp in milliseconds.

3.2.2.4.1 Health Data Callback Overview

Health data	Callback	Date object	Detail object	Description
Steps	`onSyncStep`	`StepSyncBean`	`StepItemBean`	Today's and historical steps may be returned in two callbacks
Sleep	`onSyncSleep`	`SleepSyncBean`	`SleepItemBean`	Returns multiple days of sleep data stored on the device
Heart rate	`onSyncHr`	`HeartRateSyncBean`	`HeartRateItemBean`	Grouped by date
Blood pressure	`onSyncBp`	`BloodPressSyncBean`	`BloodPressItemBean`	Grouped by date
Blood oxygen	`onSyncBo`	`BloodOxySyncBean`	`BloodOxyItemBean`	Grouped by date
Body temperature	`onSyncTemp`	`BodyTempSyncBean`	`BodyTempItemBean`	Grouped by date
Stress	`onSyncPressure`	`PressureSyncBean`	`PressureItemBean`	Grouped by date
Blood sugar	`onSyncBloodSugar`	`BloodSugarSyncBean`	`BloodSugarItemBean`	Grouped by date
HRV	`onSyncHrv`	`HrvSyncBean`	`HrvItemBean`	Grouped by date
Dhikr count	`onSyncMuslimCount`	`MuslimCountSyncBean`	`MuslimCountItemBean`	Includes the daily total and hourly details

Except for steps, sleep, and Dhikr count, the date objects share the following basic structure:

Field	Type	Description
time	long	Date timestamp in Unix seconds
itemCount	int	Number of details for the day
items	List	Measurement details for the day

3.2.2.4.2 Standard Measurement Details

Standard measurement details use `timeMills` as the measurement time in Unix seconds. Their value fields are listed below:

Data	Detail object	Value field	Unit/conversion
Heart rate	`HeartRateItemBean`	`hr`	bpm
Blood pressure	`BloodPressItemBean`	`sp` (systolic), `dp` (diastolic)	mmHg
Blood oxygen	`BloodOxyItemBean`	`bloodOxy`	%
Body temperature	`BodyTempItemBean`	`temp`	Actual temperature = `temp / 10` °C; for example, 365 means 36.5°C
Stress	`PressureItemBean`	`pressure`	Device stress value, unitless
Blood sugar	`BloodSugarItemBean`	`sugar`	`float`; do not process it as an integer
HRV	`HrvItemBean`	`hrv`	ms

> **Caution**
>
> Blood pressure contains two values, `sp` and `dp`, and must not be processed as a single value. Blood sugar `sugar` is a `float` and must not be processed as an integer.

3.2.2.4.3 Step Data

Callback: `onSyncStep(List<StepSyncBean> data)`

> **Caution**
>
> If the device contains historical step data, `onSyncStep` is called twice. The first callback returns today's steps and normally contains one `StepSyncBean`. The second callback returns historical steps and may contain multiple dates.

Both today's and historical step data use `StepSyncBean`:

Field	Type	Description
time	long	Date timestamp in Unix seconds
totalSteps	int	Total steps for the day
totalCalorie	int	Total calories for the day, cal
totalDistance	int	Total distance for the day, m
itemCount	int	Number of details
activityDataInterval	int	Step detail interval in minutes; defaults to 60 when unconfigured
items	`List<StepItemBean>`	Actual step details returned by the device

`StepItemBean` fields:

Field	Type	Description
timestamp	long	Detail timestamp in Unix seconds
index	int	Detail index within the day at the current step interval
steps	int	Steps in this detail
calorie	int	Calories in this detail
distance	int	Distance in this detail

Data	`items` content	Source of daily totals	Detail interval
Today	Actual details returned by the device for today	Uses the total steps, calories, and distance returned by the device	Determined by `activityDataInterval`; supports 10 or 60 minutes
History	Actual device details grouped by date	Sums the steps, calories, and distance in that day's historical details	Fixed at 60 minutes

`activityDataInterval=60` means one detail per hour, and `10` means one detail every 10 minutes. Use `StepItemBean.timestamp` as the exact detail time.

3.2.2.4.4 Sleep Data

Callback: `onSyncSleep(List<SleepSyncBean> data)`

> **Caution**
>
> Returns sleep data stored on the device, with a separate record for each night sleep or nap session. Data may span multiple days. Nap support requires compatible firmware.

`SleepSyncBean` fields:

Field	Type	Description
time	long	Sleep record timestamp in Unix seconds
totalSleepTime	long	Total sleep duration in minutes
asleepTime	long	Sleep start timestamp in Unix seconds
awakeTime	long	Wake-up timestamp in Unix seconds
itemCount	int	Number of sleep state details
items	`List<SleepItemBean>`	Sleep state details
sleepCategory	int	Sleep category: `0` night sleep, `1` nap

`SleepItemBean` fields:

Field	Type	Description
len	int	Duration of the current sleep state in minutes
sleepType	int	Sleep state: 0. awake, 1. light sleep, 2. deep sleep, 3. REM
isTemporary	int	Data status: 0. final data, 1. temporary data

3.2.2.4.5 Dhikr Count Data

Callback: `onSyncMuslimCount(List<MuslimCountSyncBean> data)`

`MuslimCountSyncBean` fields:

Field	Type	Description
time	long	Date timestamp in Unix seconds
itemCount	int	Number of details for the day
totalCount	int	Total count for the day
items	`List<MuslimCountItemBean>`	Hourly count details for the day

`MuslimCountItemBean` fields:

Field	Type	Description
timeMills	long	Detail timestamp in Unix seconds
count	int	Cumulative count for the corresponding hour
date	String	Detail date
hour	String	Detail hour

3.2.3 OTA Upgrade

> **Caution**

> The OTA firmware file must be provided by the device manufacturer. Before upgrading, follow 3.2.1.3 Get Device Information to read `FirmVersionBean.deviceClazz` and compare it with the device model supported by the firmware file. Upgrade only when the two models match exactly. Do not upgrade when they do not match, as using firmware for another model may make the device unusable.

3.2.3.1 Get Available Firmware

Use the following endpoint to query the available firmware list for a device model:

```
GET https://ruiwo168.com/api/device/getOtaListByModel?model=<deviceClazz>
```

The `model` query parameter corresponds to `FirmVersionBean.deviceClazz` returned by `getFirmwareVersionJL()`. Read the device firmware information first and use the actual `deviceClazz` reported by the device.

```
val firmwareCallback = object : FirmwareCallback {
  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    DHBleSdk.dispose(this)
    Log.e("OTA", "firmware info get failed, errorCode=$errorCode")
  }

  override fun onResult(data: FirmVersionBean?) {
    DHBleSdk.dispose(this)

    val deviceClazz = data?.deviceClazz.orEmpty()
    val deviceNo = data?.deviceNo.orEmpty()
    if (deviceClazz.isBlank()) {
      Log.e("OTA", "deviceClazz is empty")
      return
    }

    // Use the app's existing networking component to request this URL on a background thread.
    val url = "https://ruiwo168.com/api/device/getOtaListByModel?model=$deviceClazz"
    Log.d("OTA", "query firmware: $url, currentVersion=$deviceNo")
  }
}

DHBleSdk.subscribeData(firmwareCallback)
DHBleSdk.getFirmwareVersionJL()
```

Example response:

```
{
  "code": 0,
  "msg": "操作成功",
  "data": [
    {
      "deviceModel": "DEVICE_MODEL",
      "toVersion": "X.Y.Z",
      "size": 123456,
      "downloadUrl": "https://example.com/path/firmware.bin"
    }
  ]
}
```

Only the following fields in `data` are required for the OTA flow. Other fields may be ignored:

Field	Type	Description
deviceModel	String	Target device model; it must exactly match `FirmVersionBean.deviceClazz`
toVersion	String	Firmware version represented by `downloadUrl`; used to determine whether a newer release is available
size	Int	Firmware file size in bytes
downloadUrl	String	Firmware download URL

For production releases, compare the current `deviceNo` with the target `toVersion` numerically by each `X.Y.Z` segment and normally prompt only for a newer version. Do not compare versions as plain strings. For testing, the same version or a downgrade may be installed after confirming that the firmware is valid.

Before downloading and again before upgrading, verify that `deviceModel` exactly matches the device's `deviceClazz`. After downloading the firmware to local storage, pass its file path to `ringOtaWithFileData()`. When hosting firmware on your own server, maintain the mapping between device models, version numbers, and firmware files.

3.2.3.2 Perform OTA Upgrade

Pre-upgrade validation:

Check	Data source	Requirement
Current device model	`FirmVersionBean.deviceClazz`	Read using `getFirmwareVersionJL()`
Firmware file target model	Provided by the device manufacturer	Must exactly match the current device's `deviceClazz`
Current firmware version	`FirmVersionBean.deviceNo`	Can be used to determine whether an upgrade is required

Method Description:

```
fun ringOtaWithFileData(filePath: String, callback: OnFileTransferCallback)
```

> **Tip**
> **Firmware file:** For Nordic devices, pass the ZIP firmware package path without extracting it. For other platforms, pass the corresponding firmware file path.

> **AAR selection:** Choose `nordic` for Nordic OTA; otherwise, choose `basic`. For dependency configuration, see “Step 2: Add the SDK and its dependencies.”

Parameter Description:

Parameter	Type	Description
filePath	String	Firmware file path
callback	OnFileTransferCallback	Transfer progress callback

Example of usage:

```
val otaPath = ""        // Firmware file provided by the device manufacturer
val otaDeviceClazz = "" // Device model supported by the firmware file

fun startOta() {
  DHBleSdk.ringOtaWithFileData(otaPath, object : OnFileTransferCallback {
    override fun onProgress(pro: Float) {
      Log.e("OTA", "progress: $pro")
    }

    override fun onFinish() {
      Log.e("OTA", "OTA finish")
    }

    override fun onFail(code: Int) {
      Log.e("OTA", "OTA fail: $code")
    }
  })
}

val firmwareCallback = object : FirmwareCallback {
  override fun onSuccess() {
  }

  override fun onFail(errorCode: Int) {
    DHBleSdk.dispose(this)
    Log.e("OTA", "firmware info get failed, errorCode=$errorCode")
  }

  override fun onResult(data: FirmVersionBean?) {
    DHBleSdk.dispose(this)

    val deviceClazz = data?.deviceClazz.orEmpty()
    if (deviceClazz.isBlank() || otaDeviceClazz.isBlank() || deviceClazz != otaDeviceClazz) {
      Log.e("OTA", "device model mismatch: device=$deviceClazz, firmware=$otaDeviceClazz")
      return
    }

    startOta()
  }
}

// Get device information first. Start OTA only after the device model matches.
```

```
DHBleSdk.subscribeData(firmwareCallback)
DHBleSdk.getFirmwareVersionJL()
```

3.2.4 Sport Workout

> **Caution**
>
> The configuration table property supporting multiple sports is `isNewSport`. After enabling multi-sport mode, the device will enter exercise mode. Neither disconnecting the app nor closing it will stop the activity; it can only be stopped manually through the app or the device itself. Therefore, for devices with multi-sport functionality, please check the status after connecting to determine if it is currently in exercise mode, as this may affect the use of other functions.
>
> **The exercise duration must exceed 2 minutes for the device to save the workout data.**

3.2.4.1 Obtain the device's multiple motion states.

> Check if the device is currently engaged in multiple activities; only start a new activity if it's not currently engaged in any activity.
>
> Subscribe to `SportGetControlCallback` to receive the results.

Method Description:

`fun controlGetSportJLData()`

Parameter Description:

WorkoutControlType Enum	Type	Description	Value
Workout_Begin	Int	Integer	Start workout
Workout_Continue	Int	Integer	Continue workout
Workout_Pause	Int	Integer	Pause workout
Workout_Finish	Int	Integer	End workout

Example of usage:

```
private val sportGetCallback by lazy {
  object : SportGetControlCallback {
    override fun onSuccess() {
      Log.e("RWSDK", "SportGetControlCallback onSuccess")
    }

    override fun onFail(errorCode: Int) {

    }

    override fun onResult(data: NewSportBean) {
```

```
        val bleActivityMode = data.sportType
        val tControlType = data.status

        Log.e("RWSDK", "SportGetControlCallback onResult " + bleActivityMode + " " + tControlType)

        //0x01 Start 0x03 Pause 0x02 Continue 0x04 End
        if (tControlType?.isInRunning == true) {
          // In workout, directly enter the workout activity

          val intent = Intent(this@WorkoutTypeActivity, WorkoutRunningActivity::class.java).apply {
            putExtra("bleActivityMode", bleActivityMode?.value)
            putExtra("controlType", tControlType.value)
          }
          startActivity(intent)
        }
        else{
          if (isItemClick){
            isItemClick = false

            DHBleSdk.subscribeData(sportControlCallback)
            DHBleSdk.controlSportJL(mBleActivityMode!!,
                                    WorkoutControlType.Workout_Begin)
          }
        }
      }
    }
  }

DHBleSdk.subscribeData(sportGetCallback)
DHBleSdk.controlGetSportJLData()
```

3.2.4.2 Control the device to enter multi-sport mode

> The control device enters multi-motion mode and starts the motion.
>
> Changes in sports data are received through subscriptions via `SportDataPushCallback` notifications.
>
> Subscribe to `SportControlCallback` to receive the results.

Method Description:

`fun controlSportJL(sportType: BleActivityMode, sportStatus: WorkoutControlType)`

Parameter Description:

Parameter	Type	Description	Value
sportType	BleActivityMode	Sport Type	sportType: Refer to BleActivityMode
sportStatus	WorkoutControlType	Sport Status	sportStatus: Start, Pause, Resume, End; Refer to WorkoutControlType

SportDataPushCallback Sport data change notification return data SportDataPushBean description:

SportDataPushBeanParameter	Type	Description	Value
ActivityTime	Int	Integer	Duration of exercise, unit seconds (s);
ActivitySteps	Int	Integer	Number of steps during exercise
ActivityDistance	Int	Integer	Distance covered during exercise, unit meters (m);
ActivityCalorie	Int	Integer	Calories burned during exercise, unit calories (cal);
ActivityHr	Int	Integer	Dynamic heart rate during exercise

Example of usage:

```
DHBleSdk.subscribeData(sportControlCallback)
DHBleSdk.controlSportJL(mBleActivityMode!!,
                        WorkoutControlType.Workout_Begin)


private val sportRealPushCallback by lazy {
  object : SportDataPushCallback {
    override fun onSuccess() {
      Log.e("RWSDK", "SportDataPushCallback onSuccess")
    }

    override fun onFail(errorCode: Int) {
      Log.e("RWSDK", "SportDataPushCallback onFail: $errorCode")
    }

    override fun onResult(data: SportDataPushBean) {
      runOnUiThread {
        // Update sport type and control status
        updateData(data)
      }
    }
  }
}

// Subscribe to real-time sport data
DHBleSdk.subscribeData(sportRealPushCallback)
```

Note: The corresponding names for BleActivityMode can be found in the example Demo strings.xml file: `<string-array name="jlrunning_string_array">`

3.2.4.3 Control enabling/disabling real-time sport data notifications from the device

> Control the real-time notification of motion data when the device is turned on/off;
>
> Changes in sports data are received through subscriptions via `SportDataPushCallback` notifications. Sometimes, when the app is closed or running in the background, you can instruct the device to stop sending data notifications.

Method Description:

`fun setExerciseMore(type: Int)`

`fun setExerciseMore(type: Int, callback: CustomStatusCallback)`

The setting result is returned via `CustomStatusCallback`; real-time workout data is received via `SportDataPushCallback.onResult()`. Call `dispose(callback)` after the result is handled or when leaving the page.

Parameter Description:

Parameter	Type	Description	Value
type	Int	Integer	1: Enable notification of exercise data; 0: Disable notification of exercise data;

Example of usage:

```
// Exit exercise interface
DHBleSdk.setExerciseMore(0, object : CustomStatusCallback {
    override fun onSuccess() {
        DHBleSdk.dispose(this)
        Log.e("RWSDK", "Workout reporting disabled")
    }

    override fun onFail(errorCode: Int) {
        DHBleSdk.dispose(this)
        Log.e("RWSDK", "setExerciseMore failed: $errorCode")
    }
})
```

3.2.4.4 Get Multi-Sport Data Report

> Subscribe to `Sport3ResultCallback` to receive the results.

Method Description:

`fun getSport3ResultJL()`

return SportResultBean Parameter Description:

SportResultBean Parameter	Type		Value
startTime	long		Exercise start time timestamp, in seconds (s)
exerciseTime	long		Exercise duration, in seconds (s)
workModel	BleActivityMode		Type of exercise
step	Int		Number of steps, unit: steps
distance	Int		Distance, meters (m)
calorie	Int		calories, cal
viewType	Int		Current exercise types include/exclude steps, cadence, pace, and distance: With cadence: viewTypeHaveStepFaq: Without step count: viewTypeNoStepNum: With pace: viewTypeHavePace: Without distance: viewTypeNoDistance:
newSportHrs	Array		The current list of heart rates generated during exercise, saved every minute;
pacePerKmList	Array		Pace per kilometer list, unit: seconds/km; null if device does not support
.....			Other attributes can be found in the class documentation.

Example of usage:

```
private val sportResult3Callback by lazy {
  object : Sport3ResultCallback {
    override fun onSuccess() {
      Log.e("RWSDK", "Sport3ResultCallback onSuccess")
    }

    override fun onFail(errorCode: Int) {
      Log.e("RWSDK", "Sport3ResultCallback onFail " + errorCode)
      finish()
    }

    override fun onResult(data: List<SportResultBean>) {
      Log.e("RWSDK", "Sport3ResultCallback onResult " + data.size)
      //Save Data
      finish()
    }
  }
}
```

3.2.5 AI Recording

> Recording control, status query and recording file management (list/download/delete/format) for smart recording rings;
>
> Configuration table attribute: `isSupportRecording`; requires MIC hardware support.

3.2.5.1 Start/Stop Recording

> Control the device to start or stop recording; the device stores the file locally after recording starts.
>
> The result is returned via [callback].

Method Description:

`fun recordControl(start: Boolean, callback: RecordControlCallback)`

Parameter Description:

Parameter	Type	Description	Value
start	Boolean	Boolean	true: start recording; false: stop recording;
callback	RecordControlCallback	Result callback	onResult: recording status after the operation (see table below); onFail: failed;

`RecordControlCallback.onResult` returns the recording status after the operation:

Value	Description
0x01	Recording
0x00	Idle

Example of usage:

```
DHBleSdk.recordControl(true, object : RecordControlCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "record control failed: $errorCode")
    }
    override fun onResult(data: Int) {
        Log.e("RWSDK", "record control result=$data") //0x01 recording, 0x00 idle; query full status afterwards (3.2.5.2)
    }
})
```

3.2.5.2 Query Recording Status

> Query whether the device is currently recording and the recording storage capacity; it is recommended to query once after the app connects to confirm the device recording status.
>
> The result is returned via [callback]'s onResult(RecordStatusBean).

Method Description:

`fun getRecordStatus(callback: RecordStatusCallback)`

Parameter Description:

Parameter	Type	Description	Value
callback	RecordStatusCallback	Result callback	onResult: RecordStatusBean (see table below); onFail: failed;

RecordStatusBean parameters:

RecordStatusBean	Type	Description	Value
status	Int	Integer	0x01: recording; 0x00: idle;
recording	Boolean		Whether recording
startTime	long		Start timestamp of this recording, in seconds (s), only while recording;
duration	long		Recorded duration, in seconds (s), only while recording;
totalCapacity	long		Total recording storage capacity, in bytes (B);
remainingCapacity	long		Remaining recording storage capacity, in bytes (B);
realtime	Boolean		true: real-time recording; false: non-real-time recording

Example of usage:

```
DHBleSdk.getRecordStatus(object : RecordStatusCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "record status failed: $errorCode")
    }
    override fun onResult(data: RecordStatusBean) {
        Log.e("RWSDK", "recording=${data.isRecording} duration=${data.duration}s"
                + " remaining=${data.remainingCapacity}/${data.totalCapacity}")
    }
})
```

Recording status real-time push: the push type is `DevicePushType.RECORD_STATUS` and `value` is a `RecordStatusBean` (isRecording/status/startTime/duration/capacity). See 3.2.1.21 Device Push Notification for how to subscribe.

3.2.5.3 Get Recording File List

> Get the metadata of all recording files on the device; the device returns pages, the SDK stitches them automatically, and the complete list is returned once via [callback]'s onResult after everything is received.

Method Description:

`fun getRecordFileList(callback: RecordFileListCallback)`

Parameter Description:

Parameter	Type	Description	Value
callback	RecordFileListCallback	Result callback	onResult: complete file list (see table below); onFail: failed;

RecordFileItemBean parameters:

RecordFileItemBean	Type	Description	Value
fileId	long		File ID, used for download (3.2.5.4) / delete (3.2.5.5);
fileSize	long		File size, in bytes (B);
duration	long		Recording duration, in seconds (s);
timestamp	long		Recording time as a Unix timestamp, in seconds (s);

Example of usage:

```
DHBleSdk.getRecordFileList(object : RecordFileListCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "file list failed: $errorCode")
    }
    override fun onResult(data: MutableList<RecordFileItemBean>) {
        Log.e("RWSDK", "file count=${data.size}")
        data.forEach {
            Log.e("RWSDK", "fileId=${it.fileId} size=${it.fileSize} duration=${it.duration}s")
        }
    }
})
```

3.2.5.4 Download Recording File

> Download a single recording file by file ID. Progress is reported during transmission, and `complete=true` means `fileData` holds the complete file data.
>
> Progress and the final file data are returned via [callback]'s onResult(RecordFileTransferBean) in multiple frames.
>
> **Note:** Transmission is sequential; resume from breakpoint is not supported - on failure the entire file must be downloaded again; repeated calls before completion are rejected with onFail; `fileData` is the raw device OPUS data and must be converted to an Ogg Opus (.opus) file via `OpusBinConverter.convert()` before playback.

Method Description:

```
fun transferRecordFile(fileId: Long, callback: RecordFileTransferCallback)
```

Parameter Description:

Parameter	Type	Description	Value
fileId	long		File ID, from the `fileId` returned by `getRecordFileList()`;
callback	RecordFileTransferCallback	Result callback	onResult: multiple frames (see table below), complete=true is the final frame; onFail: failed/timeout/disconnected;

RecordFileTransferBean parameters:

RecordFileTransferBean	Type	Description	Value
fileType	Int	Integer	0x01: recording;
duration	long		Recording duration, in seconds (s);
timestamp	long		Recording time as a Unix timestamp, in seconds (s);
fileId	long		File ID;
format	Int	Integer	0x02: OPUS;
fileSize	long		Total file size, in bytes (B);
received	long		Received bytes, in bytes (B);
progress	float		Transfer progress 0~1;
complete	Boolean		Whether the transfer is complete; true means fileData is the complete file data;
completedAt	long		Completion timestamp, in seconds (s);
filePath	String		App-side save path, assigned by the caller;
fileData	byte[]		Complete raw OPUS data when complete=true;

Example of usage:

```
DHBleSdk.transferRecordFile(fileId, object : RecordFileTransferCallback { //fileId from the file list
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "transfer failed: $errorCode")
    }
    override fun onResult(data: RecordFileTransferBean) {
        val percent = (data.progress * 100).toInt()
        Log.e("RWSDK", "transfer ${data.received}/${data.fileSize} bytes ($percent%)")
        if (data.isComplete) {
            //Convert raw OPUS data to an Ogg Opus file and save
            val opusBytes = OpusBinConverter.convert(data.fileData ?: ByteArray(0))
            val file = File(dir, "${data.fileId}_${data.duration}_${data.completedAt}.opus")
            file.writeBytes(opusBytes)
        }
    }
})
```

3.2.5.5 Delete Recording File

> Delete a single recording file on the device by file ID.
>
> The result is returned via [callback].

Method Description:

`fun deleteRecordFile(fileId: Long, callback: RecordFileDeleteCallback)`

Parameter Description:

Parameter	Type	Description	Value
fileId	long		File ID, from the `fileId` returned by `getRecordFileList()`;
callback	RecordFileDeleteCallback	Result callback	onResult: status code (see table below); onFail: failed;

`RecordFileDeleteCallback.onResult` returns a status code:

Value	Description
0x00	Success
Other	Failure

Example of usage:

```
DHBleSdk.deleteRecordFile(fileId, object : RecordFileDeleteCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "delete failed: $errorCode")
    }
    override fun onResult(data: Int) {
        Log.e("RWSDK", "delete result=$data") //0x00 success
    }
})
```

3.2.5.6 Format Recording Storage

> Clear the device recording storage, **deleting all recording files on the device, unrecoverable; confirm twice before calling.**
>
> The result is returned via [callback].

Method Description:

`fun formatRecordStorage(callback: RecordFormatCallback)`

Parameter Description:

Parameter	Type	Description	Value
callback	RecordFormatCallback	Result callback	onResult: status code (see table below); onFail: failed;

`RecordFormatCallback.onResult` returns a status code:

Value	Description
0x00	Success
Other	Failure

Example of usage:

```
DHBleSdk.formatRecordStorage(object : RecordFormatCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "format failed: $errorCode")
    }
    override fun onResult(data: Int) {
        Log.e("RWSDK", "format result=$data") //0x00 success
    }
})
```

3.2.5.7 Real-time Recording

> Requires `isSupportRecording` and device firmware that supports real-time recording.

`fun recordRealtimeControl(start: Boolean, callback: RecordControlCallback)`

Parameter / callback	Description
start	`true`: start real-time recording; `false`: stop
callback	`onResult(Int)` returns the resulting recording state (1 recording, 0 idle), and `onFail(errorCode)` reports failure. Audio is not returned through this callback.

Audio arrives through `OnDevicePushListener` as `DevicePushType.RECORD_REALTIME_DATA`. Its `value` is a `RecordRealtimeDataBean`:

Property	Type	Description
sequence	Int	Packet sequence 0–255, wrapping from 255 to 0; use it to check packet loss and ordering
audioData	ByteArray	Raw Opus-compressed audio data for this packet

```
val recordingListener = object : OnDevicePushListener {
    override fun onPush(data: PushData) {
        if (data.type != DevicePushType.RECORD_REALTIME_DATA) return
        val packet = data.value as? RecordRealtimeDataBean ?: return
        // Process packet.audioData in arrival order. Do not perform slow work in this callback.
        Log.e("RWSDK", "audio sequence=${packet.sequence}, size=${packet.audioData.size}")
    }
}
DHBleSdk.addOnDevicePushListener(recordingListener)

val controlCallback = object : RecordControlCallback {
    override fun onSuccess() = Unit
    override fun onResult(data: Int) { Log.e("RWSDK", "recording state=$data") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "record control failed=$errorCode") }
}
DHBleSdk.recordRealtimeControl(true, controlCallback)
// To stop: DHBleSdk.recordRealtimeControl(false, controlCallback)
// When leaving the page or no longer needed: DHBleSdk.removeOnDevicePushListener(recordingListener)
```

Register the listener before starting. Recording status remains available through `RECORD_STATUS` pushes or `getRecordStatus()`; `realtime` indicates whether real-time mode is active.

5.2.5 Sensor Raw Data

This section covers two different data retrieval methods:

Data	Retrieval method	Description
PPG/ACC/PPG Red/IR raw data	History retrieval	The APP starts and stops collection, then actively synchronizes the stored data
Sleep state data	Real-time push	The device automatically pushes data during sleep; the APP only needs to subscribe to the callback

> **Important**
>
> PPG/ACC/PPG Red/IR raw data does not support real-time push and is available only through history retrieval. Sleep state data uses only real-time push and is not retrieved through the historical raw data API.
>
> The device stores only the latest PPG raw-data record, containing approximately one minute of data, whether collection is triggered by scheduled monitoring or a manual measurement. A subsequent collection may overwrite data that has not yet been synchronized. When collection finishes, the device reports the stop through `SensorRawControlCallback.onResult(reason)`. Call `ringGetHistorySensorRaw()` promptly to synchronize and persist the data.
>
> Function table properties: `isSupportSensorRawPPG` (PPG), `isSupportSensorRawACC` (ACC), `isSupportSensorRawPPGRed` (PPG Red), `isSupportSensorRawIR` (IR), and `isSupportSensorRawSleep` (sleep real-time data).

Valid `sensorType` combinations for PPG/ACC/PPG Red/IR historical collection:

Value	Meaning	Description
1	ACC	ACC only
2	PPG Green	PPG Green only
3	PPG Green + ACC	PPG Green and ACC simultaneously
4	PPG Red	PPG Red only
5	PPG Red + ACC	PPG Red and ACC simultaneously
10	PPG Green + IR	PPG Green and IR simultaneously
11	PPG Green + ACC + IR	PPG Green, ACC and IR simultaneously
12	PPG Red + IR	PPG Red and IR simultaneously
13	PPG Red + ACC + IR	PPG Red, ACC and IR simultaneously

> **Rules: PPG Green and PPG Red cannot coexist; IR cannot start alone, must be combined with PPG Green or PPG Red.**
>
> **Note:** The control API's `sensorType` is a sensor bitmask, while the returned object's `type` is a data type. They use different numbering. For example, `sensorType=1` starts ACC, while historical data `type=1` means PPG. `sensorType=5` starts PPG Red + ACC, while sleep real-time data `type=5` means a sleep state.

5.2.5.0 PPG Timed Monitoring

> PPG timed monitoring setting, similar to heart rate/HRV timed monitoring;
>
> Configuration table attribute: `isSupportPPGMonitoring`
>
> The result is returned via `ReminderSettingCallback`.

Method Description:

`fun setHealthMonitor(type: HealthMonitorType, monitorBean: HealthMonitorBean, callback: ReminderSettingCallback)` — type is fixed `HealthMonitorType.PPG`

`fun getHealthMonitor(type: HealthMonitorType, callback: ReminderSettingCallback)` — type is fixed `HealthMonitorType.PPG`

Parameter Description:

Parameter	Type	Description	Value
type	HealthMonitorType	Enum	Fixed `HealthMonitorType.PPG`
monitorBean	HealthMonitorBean	Entity	isOpen: true on/false off remindDuration: default interval 30 minutes startHour: 0 fixed startMin: 0 fixed endHour: 23 fixed endMin: 59 fixed
callback	ReminderSettingCallback	Interface	Set result: onSuccess()/onFail(); Query result: onResult(DrinkReminderBean);

Example of usage:

```
//Set PPG monitoring
val healthMonitorBean = HealthMonitorBean()
healthMonitorBean.isOpen = true
healthMonitorBean.remindDuration = 60
healthMonitorBean.startHour = 0
healthMonitorBean.startMin = 0
healthMonitorBean.endHour = 23
healthMonitorBean.endMin = 59
DHBleSdk.setHealthMonitor(HealthMonitorType.PPG, healthMonitorBean, object : ReminderSettingCallback {
    override fun onSuccess() { /* set success */ }
    override fun onFail(errorCode: Int) { /* set failed */ }
    override fun onResult(data: DrinkReminderBean?) { /* query result (with getHealthMonitor) */ }
})

//Get PPG monitoring configuration
DHBleSdk.getHealthMonitor(HealthMonitorType.PPG, object : ReminderSettingCallback {
    override fun onSuccess() = Unit
    override fun onFail(errorCode: Int) { /* query failed */ }
    override fun onResult(data: DrinkReminderBean?) { /* current PPG monitoring configuration */ }
})
```

5.2.5.1 Start and Stop Sensor Raw Data

> This API controls only PPG/ACC/PPG Red/IR historical raw data collection. It is not required for sleep real-time data.
>
> Subscribe to `SensorRawControlCallback` to receive start/stop results;
>
> The device may also stop the sensor actively, notified via `SensorRawControlCallback.onResult(reason)`, where reason is the stop cause (1 byte).

Method Description:

`fun ringControlSensorRaw(outputType: Int, sensorType: Int)`

Parameter Description:

Parameter	Type	Description	Value
outputType	Int	Output control type	1: Start Sensor output 2: Stop Sensor output
sensorType	Int	Sensor type (bitmask)	See valid combinations table above

Example of usage:

```
//Subscribe to control callback
DHBleSdk.subscribeData(sensorRawControlCallback)

//Start PPG+ACC raw data output (sensorType=3)
DHBleSdk.ringControlSensorRaw(1, 3)

//Stop PPG+ACC raw data output
DHBleSdk.ringControlSensorRaw(2, 3)

//Listen for control results and device-initiated stop
private val sensorRawControlCallback by lazy {
    object : SensorRawControlCallback {
        override fun onResult(data: Int?) {
            Log.e("RWSDK", "device stopped sensor, reason=" + data)
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() {
            Log.e("RWSDK", "sensor control command OK")
        }
    }
}
```

5.2.5.2 Historical Raw Data Retrieval

> PPG/ACC/PPG Red/IR raw data is available only through history retrieval. The device collects and stores the data first, and the APP later actively synchronizes it using `ringGetHistorySensorRaw()`.
>
> Subscribe to `SensorHistoryRawCallback` to receive data. `onResult` returns the historical raw data list, and `onSuccess` indicates that synchronization is complete.

Method Description:

`fun ringGetHistorySensorRaw()`

SensorHistoryRawBean fields:

Field	Type	Description
type	int	Data type: 1=PPG, 2=ACC, 3=PPG Red, 4=IR
sequence	int	Packet sequence starting from 1 and incrementing for every returned packet; all enabled sensors share the same sequence
ppgDataList	List<Integer>	PPG data list; each item is int32
accDataList	List<AccRawItem>	ACC data list; each item contains x, y, and z (int16)
ppgRedDataList	List<Integer>	PPG Red data list; each item is int32
irDataList	List<Integer>	IR data list; each item is int32

AccRawItem fields:

Field	Type	Description
x	int	X-axis (int16)
y	int	Y-axis (int16)
z	int	Z-axis (int16)

> `onResult` returns `List<SensorHistoryRawBean>` containing all historical sensor records.

Example of usage:

```
DHBleSdk.subscribeData(sensorHistoryRawCallback)
DHBleSdk.ringGetHistorySensorRaw()

private val sensorHistoryRawCallback by lazy {
    object : SensorHistoryRawCallback {
        override fun onResult(data: List<SensorHistoryRawBean>?) {
            data?.let {
                Log.e("RWSDK", "SensorHistory count=${it.size}")
                for (bean in it) { Log.e("RWSDK", "  " + bean.toString()) }
            }
        }
        override fun onFail(errorCode: Int) {}
        override fun onSuccess() { Log.e("RWSDK", "SensorHistory sync finished") }
    }
}
```

5.2.5.3 Sleep State Real-Time Push

> Sleep state data supports only real-time push. Do not call `ringControlSensorRaw()` to start or stop it. When supported by the device, sleep state data is automatically pushed during sleep.
>
> Function table property: `isSupportSensorRawSleep`.
>
> Subscribe to `SensorRawDataCallback` to receive sleep state data. The returned object is `SensorRawDataBean`.

Return Value:

SensorRawDataBean field	Type	Description
type	int	Fixed at 5, indicating sleep state data
sleepDataList	List<long[]>	Sleep state list; each item contains [0]=Unix timestamp in seconds and [1]=sleep mode

Sleep modes:

Value	Description
17	Sleep start
34	Sleep end
1	Deep sleep
2	Light sleep
3	Awake
4	REM

Example of usage:

```
private val sleepRawDataCallback = object : SensorRawDataCallback {
    override fun onResult(data: SensorRawDataBean?) {
        if (data?.type == 5) {
            data.sleepDataList.forEach { item ->
                val timestamp = item[0]
                val sleepMode = item[1]
                Log.e("RWSDK", "sleep timestamp=$timestamp mode=$sleepMode")
            }
        }
    }

    override fun onFail(errorCode: Int) {
        Log.e("RWSDK", "sleep data failed, errorCode=$errorCode")
    }

    override fun onSuccess() {
    }
}

// Call during initialization.
fun registerSleepRawDataCallback() {
    DHBleSdk.subscribeData(sleepRawDataCallback)
}

// Call when sleep data is no longer needed.
fun unregisterSleepRawDataCallback() {
    DHBleSdk.dispose(sleepRawDataCallback)
}
```

5.2.5.4 Set and Get PPG Sample Rate

> Requires `isSupportPpgSampleRate`. Set or read the PPG sample rate in Hz, for example 100 Hz or 200 Hz. Supported values depend on the device firmware.

```
fun setPpgSampleRate(sampleRate: Int, callback: CustomStatusCallback)
fun getPpgSampleRate(callback: PpgSampleRateCallback)
```

Parameter/Callback	Description
sampleRate	Sample rate in Hz. Supported values depend on the device firmware
CustomStatusCallback	`onSuccess()` when saved; `onFail(errorCode)` on failure
PpgSampleRateCallback	`onResult(Int)` returns the current sample rate in Hz; `onFail(errorCode)` on failure

```
DHBleSdk.setPpgSampleRate(100, object : CustomStatusCallback {
    override fun onSuccess() { Log.e("RWSDK", "PPG sample rate saved") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "set failed=$errorCode") }
})

DHBleSdk.getPpgSampleRate(object : PpgSampleRateCallback {
    override fun onSuccess() = Unit
    override fun onResult(data: Int) { Log.e("RWSDK", "PPG sample rate=$data Hz") }
    override fun onFail(errorCode: Int) { Log.e("RWSDK", "get failed=$errorCode") }
})
```

SDK Revision History

V2.0.0_20261105 (2026.11.05)

- Added nap support and the `sleepCategory` field to distinguish night sleep from naps (3.2.2.4.4).

V2.0.0_20261104 (2026.11.04)

- Fixed an issue where the customer model (`customerModel`) could not be read correctly from device information (3.2.1.3).
- Added `showCustomText` for custom text display and duration settings (3.2.1.35); added `isSupportCustomTextDisplay` to the configuration table.
- Added `controlCompositeMeasurement` for multiple health measurements in one session (3.2.2.1); added `isSupportCompositeMeasurement` to the configuration table.

V2.0.0_20261028 (2026.10.28)

- Added `setPpgSampleRate` / `getPpgSampleRate` for PPG sample rate (5.2.5.4); added `isSupportPpgSampleRate` to the function table.
- Added `setCustomerModel` / `getCustomerModel` for the customer model (3.2.1.34); added `isSupportCustomerModel` to the function table and `customerModel` to device version information.

V2.0.0_20261022 (2026.10.22)

- Added `recordRealtimeControl` and real-time audio listening (3.2.5.7).
- Added counting recognition control, state queries and push notifications (3.2.1.33); added `isSupportCountControl` to the function menu.
- Added the `deviceForm` product form field to scan results and device version information (3.1.1, 3.2.1.3).

V2.0.0_20261012 (2026.10.12)

- Added SOS and repeated-tap count notifications to device push monitoring (3.2.1.21).
- Added Release events after a long press to device push monitoring: `keyType=1`, `touchType=6` (3.2.1.21).

V2.0.0_20261008 (2026.10.08)

- Added charging status to scan results via `BleDevice.getBatteryStatus()` (3.1.1).
- Updated the integration guide: Step 2 now describes selecting one of the two AAR variants (basic / nordic) and their dependencies.

V2.0.0_20260930 (2026.09.30)

- Added connected RSSI reading interface `readRssi` (3.1.7).
- Added custom vibration control interface `controlVibration` (3.2.1.31); added `isSupportVibrationControl` to the function configuration table.

V2.0.0_20260922 (2026.09.22)

- Added recording interfaces (3.2.5); added `isSupportRecording` to the function configuration table.
- Added device identity authentication interface (3.2.1.26.4); added `isSupportDeviceChallenge` to the function configuration table.
- Added sedentary reminder setting and getting interface (3.2.1.29); added `isSupportSedentary` to the function configuration table.
- Added drinking reminder setting and getting interface (3.2.1.30); added `isDrink` to the function configuration table.
- Added unified continuous monitoring setting interface `setHealthMonitor` / `getHealthMonitor` (3.2.2.2, by `HealthMonitorType`) and the single real-time detection interface `controlOpen` (3.2.2.0, session-style callback); old interfaces remain available.

V2.0.0_20260909 (2026.09.09)

- Added metric/imperial unit settings and retrieval (3.2.1.28).
- Added charging status and real-time battery monitoring (3.2.1.4).
- Lowered the minimum supported Android version to Android 7.0 (API 24).

V2.0.0_20260820 (2026.08.20)

- Added the `setDeviceTime` custom device-time API for debugging and testing.
- Added authorized password reset support (3.2.1.26.3).

V2.0.0_20260817 (2026.08.17)

- Added instant screen control (3.2.1.27).
- Fixed alarm retrieval so that `onResult` returns an empty list when the device has no alarms.
- Added the available-firmware endpoint and OTA device-model/version validation guidance (3.2.3.1).

V2.0.0_20260724 (2026.07.24)

- Added support for step-detail intervals.